Transformer-Based In-Context Learning for Unknown and Non-Concave Network Utility Maximization

Shugang Hao
Singapore University of Technology and Design
Singapore, Singapore
shugang_hao@sutd.edu.sg

Lingjie Duan*
Hong Kong University of Science and Technology
Guangzhou, China
lingjieduan@hkust-gz.edu.cn

Abstract

Network Utility Maximization (NUM) studies how to allocate network users' traffic rates under resource constraints for maximizing total network utility. Existing NUM methods largely assume users' utility functions are known and concave. In practice, however, utility functions correlating with user experience or job quality can be neither known nor concave, where renowned methods (e.g., decomposition and cross-layer optimization) easily lose convergence guarantee to the global optimum. In this work, we propose a novel transformer-based in-context learning (ICL) approach for NUM under unknown and non-concave utilities. First, we design a prompt to contain route-rate ICL example pairs and one query flow route, where the example feature/input is a flow's route vector and the example label/output is the corresponding rate allocation. Then, we use the prompt to train lightweight attention-layer parameters of the transformer for efficient ICL prediction of the query flow route's rate decision. We propose an efficient algorithm for transformer training to guarantee provable convergence to the global optimum within limited training steps. As it can be hard to collect sufficient flow data as ICL examples in practice, we extend our ICL framework to address insufficient example inputs, still guaranteeing a small utility gap to the global optimum. Experiments on Abilene network further demonstrate our method's good adaptiveness to varying utilities and great utility improvement over state-of-the-art NUM solutions with unknown and non-concave sigmoid utilities.

CCS Concepts

• **Networks** → **Network algorithms**; • **Computing methodologies** → **Machine learning**.

Keywords

Transformer, In-Context Learning, Network Utility Maximization

ACM Reference Format:
Shugang Hao and Lingjie Duan. 2018. Transformer-Based In-Context Learning for Unknown and Non-Concave Network Utility Maximization. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX).* ACM, New York, NY, USA, 20 pages. https://doi.org/XXXXXXX.XXXXXXX

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Network Utility Maximization (NUM) has been a central framework for network resource allocation and control for several decades, providing an optimization-based foundation to aim for efficiency or fairness in communication networks (e.g., [15], [37]). By formulating resource allocation as the maximization of aggregate user utility subject to network constraints, NUM has enabled systematic design and analysis of distributed algorithms for a wide range of networking problems. Applications of the NUM framework span Internet congestion control and bandwidth sharing (e.g., [15], [24], [4]), wireless rate and power control (e.g., [27], [38], [25]), cross-layer optimization in multi-hop and wireless networks (e.g., [3], [9], [37]), and scheduling and routing in stochastic networked systems (e.g., [34], [32], [26]).

Most existing work in the NUM literature assumes network users' utility functions are known and concave. Under these assumptions, extensive research has focused on solving optimal resource allocations and designing distributed algorithms with provable convergence and stability guarantees, including primal–dual congestion control (e.g., [15],[24]), decomposition-based methods [27], and layered or cross-layer optimization frameworks (e.g., [4], [3]). However, user utility functions in practical applications (e.g., edge-assisted Generative AI and compute-integrated networks) may correlate with metrics such as application-level quality of experience, age of information, compute-energy efficiency, and end-to-end inference latency, which can be neither concave nor known prior to the network system (e.g., [8], [14]). This renders existing NUM methods assuming explicitly concave utilities inapplicable or suboptimal, as underlying optimization objectives cannot be directly formulated.

In the NUM literature, there are very few studies addressing unknown utility functions. [8] solves NUM with unknown utilities by learning utility information from observed queueing delays and utility feedback. [35] studies stochastic network utility maximization with unknown threshold utilities and establishes sub-linear regret guarantees under semi-bandit feedback. [23] addresses NUM with unknown concave utilities using optimistic flow control and link scheduling based on utility feedback. However, all these approaches leverage concavity to ensure tractable optimization and performance guarantees. We only find [14] to address unknown and non-concave utilities, which studies a data-driven bilevel optimization approach to enables distributed implementation without explicit utility models. Yet, the proposed method only guarantees algorithm convergence and its theoretical gap to the global optimum remains unclear. One research question thus naturally arises:

- *Q1. How to solve the NUM problem with non-concave and unknown utility functions?*

Recently, the communication society has explored two main approaches to leverage LLMs for diverse network design tasks. One is fine-tuning pretrained models with domain-specific wireless datasets to adapt them for tasks like signal detection and network optimization (e.g., [41], [28]). However, such fine-tuning approaches typically impose prohibitive computational complexity and often lack formal performance guarantees in highly dynamic network environments. The other is in-context learning (ICL), which adapts to unseen tasks by conditioning on input–output examples (e.g., [18], [20]). By leveraging the attention mechanism for similarity-aware aggregation without requiring explicit retraining or weight updates, ICL is well suited for solving NUM problems with unknown utility functions (e.g., [6], [19]).

There are some studies introducing ICL to network design, such as transmission power allocation (e.g., [17], [42]), network deployment (e.g., [2], [30]), and network detection (e.g., [31], [39]). However, all these ICL studies are empirical and lack a theoretical foundation, providing no performance guarantees. Even in the AI society, there are very few studies on analyzing and providing theoretical guarantees of transformer-based ICL prediction (e.g., [40], [13], [21], [19]). These studies consider either binary outputs or a linear mapping function, hard to generalize to solve the complex NUM problem with non-concave utilities and coupled link capacity constraints. Our preliminary work in [12] assumes perfect data samples and is specific to a low-dimensional objective that is smooth and typically unimodal, making it restricted for NUM with noisy data and general non-concave underlying utilities. Our second research question thus arises:

- *Q2. How to properly leverage ICL for NUM and guarantee convergence to the global optimum within a limited time?*

There are two fundamental challenges in leveraging ICL for NUM. Firstly, ICL involves a non-linear transformer architecture with a softmax operation and its prediction is highly non-convex in the transformer parameters, making it challenging to optimize for a small prediction loss. Secondly, network environment changes from time to time, requiring our algorithm robust to unknown utilities.

We summarize our key novelty and main results below.

- *Transformer-based in-context learning for network utility maximization:* In this paper, we propose a novel transformer-based ICL approach for NUM. Unlike most existing work assuming known or concave utility functions, we aim for an analytical study for NUM under unknown and non-concave utilities. We provide new analytical insights into *how a network system can leverage transformer-based ICL to optimize network utility.*
- *A novel transformer-based ICL approach for NUM:* First, we design a prompt to contain route-rate ICL example pairs and one query flow route, where the example feature/input is a flow's route vector and the example label/output is the corresponding rate allocation. Then, we use the prompt to train attention-layer parameters of the transformer for efficient ICL prediction of the query flow route's rate decision
- *Theoretical guarantee to global optimum:* We propose an efficient algorithm for transformer training. Despite the highly non-convex prediction loss objective, we prove its convergence guarantee to the global optimum with limited training steps, ensuring a converged network utility loss.
- *Extension to insufficient data input:* As it can be hard to collect perfect flow data as ICL examples in practice, we extend our ICL framework to insufficient example inputs. We still manage to prove that the ICL prediction loss approximates the global optimum with limited network utility loss. Experiments on Abilene network further demonstrate our method's good adaptiveness to varying utility functions, as well as great utility improvement over state-of-the-art solutions on NUM with unknown and non-concave utility functions.

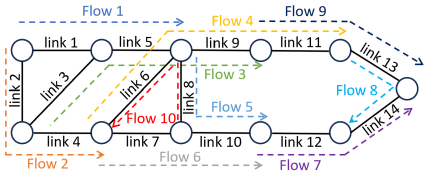

Figure 1: An example of Abilene network with $S = 10$ sources and $L = 14$ links. Flows are colored differently.

The rest of this paper is organized as follows. Section 2 introduces the system model and discusses approaches in the NUM literature. Section 3 details our transformer-based ICL framework design for NUM. Section 4 illustrates the analysis of our approach and gives theoretical guarantees. Section 5 extends to consider the insufficient data input case. Section 6 conducts experiments to verify our theoretical results. Section 8 concludes the paper.

2 System Model and Problem Formulation

In Section 2.1, we give the system model and the NUM problem formulation. In Section 2.2, we analyze the limitations of existing methods in non-concave settings to motivate our approach.

2.1 Overview of The NUM Problem

In this section, we introduce the system model and the fundamental NUM problem to solve. We consider a typical communication network represented by a set of L links and S source nodes. Each link $\ell \in [L] := \{1, \ldots, L\}$ is defined as a one-hop connection between two nodes with a capacity $c_\ell \in \mathbb{R}^+$. Each source $s \in [S] := \{1, \ldots, S\}$ is assumed to generate a single deterministic flow destined for a target node (e.g., video streaming or voice calls). This flow traverses the network via a multi-hop path consisting of a set of links $L(s) \subseteq [L]$. We associate each source with a transmission rate $x_s \in [m_s, M_s]$, where $m_s > 0$ ensures a non-zero lower bound. Figure 1 illustrates an example of Abilene network with S=10 sources and L=14 links.

Denote $S(\ell)$ as the set of sources whose unique flows traverse link ℓ. The fundamental NUM problem (e.g., [27]) aims to maximize the aggregate network utility $\sum_{s\in[S]} U_s(x_s)$ by allocating the source rates $\{x_s\}_{s\in[S]}$ subject to the link capacity constraints as follows:

$$\begin{aligned} \max_{\{x_s \in [m_s, M_s]\}_{s\in[S]}} \quad & \sum_{s\in[S]} U_s(x_s) \\ \text{subject to} \quad & \sum_{s\in S(\ell)} x_s \le c_\ell, \quad \ell = 1, \ldots, L. \end{aligned} \tag{1}$$

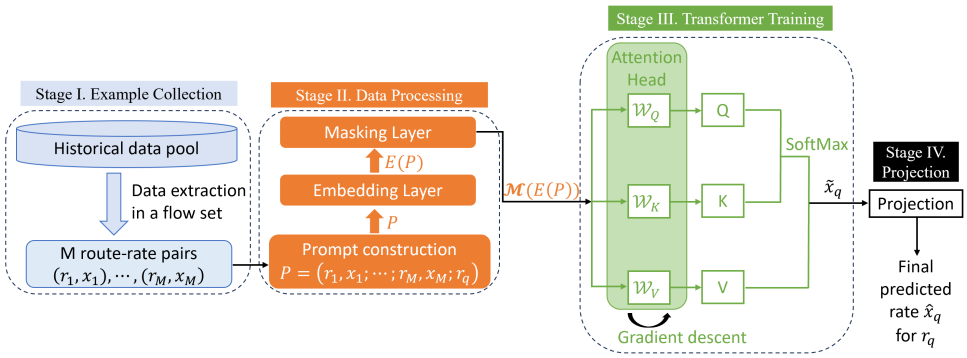

Figure 2: Overview of our transformer-based ICL framework for NUM. First, in the example collection stage, the network system collects M route-rate pairs $(r_1, x_1), \cdots, (r_M, x_M)$ of the same flow set in the historical data pool, where r_i denotes flow i's route vector and x_i denotes its source rate allocation. Then, in the data processing stage, it constructs the examples together with a query flow vector r_q into a prompt P, which will be embedded and masked before reaching the transformer. Further, in the transformer training stage, the output prompt goes through the attention heads to perform self-attention for the transformer's ICL prediction $\tilde{x}_q$, where attention parameters $(\mathcal{W}_Q, \mathcal{W}_K, \mathcal{W}_V)$ are trained for the ICL efficiency. Finally, the obtained $\tilde{x}_q$ is projected to $\hat{x}_q$, ensuring no violation of the link capacity constraints.

Note that the NUM problem in (1) constitutes the fundamental formulation for end-to-end flow control (e.g., [8], [14]). Although it does not explicitly model per-node rate adjustment in multi-hop congestion control, the end-to-end source rate allocation directly determines the traffic load on every link along each flow's multi-hop path, making it a foundational building block for joint rate control and routing optimization (e.g., [33]).

While the concavity of utility functions is a standard assumption for analytical tractability, it may not hold in practice by applications such as video streaming or delay-sensitive services (e.g., [16]). These real-world utilities frequently exhibit non-concave or sigmoidal behavior, reflecting thresholds and saturation effects. Consequently, we relax the assumption of strict concavity to consider more practical and general utility functions $U_s(\cdot)$ in the following.

Assumption 1. *Each unknown utility function $U_s \in \mathcal{U}$ is $\mathcal{K}$-Lipschitz continuous, i.e.,*

$$|U_s(x_s) - U_s(x_s')| \leq \mathcal{K} \cdot |x_s - x_s'|, \quad \forall x_s, x_s' \in \mathbb{R}^+.$$

Note that Assumption 1 on Lipschitz continuity is naturally satisfied for any concave utility function defined over the bounded domain $x_s \in [m_s, M_s]$ with $m_s > 0$. This assumption thus holds for a wide range of concave utilities (e.g., logarithmic, α-fair, or saturating linear) and non-concave functions (e.g., sigmoidal, piecewise-linear staircase, or concave–convex hybrid). Next, we show that existing studies in the NUM literature can lead to sub-optimal solutions under our relaxed Assumption 1.

2.2 Preliminary: NUM for Unknown and Non-Concave Utility Functions

As introduced in Section 1, the NUM studies with unknown utility functions (e.g., [8], [14], [23]) still rely on the concavity of (surrogate) utility functions. Their decomposition-based methods may lead to suboptimal convergence, potentially settling at local optima or even saddle points rather than the global optimum. We have the following.

Remark. For unknown and general non-concave utility functions $\{U_s\}_{s\in[S]} \in \mathcal{U}$ satisfying Assumption 1, recent NUM methodologies (e.g., [8, 14, 23]) can exhibit an optimality gap such that:

$$\sup_{\{U_s\}\in\mathcal{U}} \frac{1}{S}\left(\sum_{s\in[S]} U_s(x_s^*) - \sum_{s\in[S]} U_s(\hat{x}_s)\right) = O\left(\mathcal{K} \cdot \max_{s\in[S]} M_s\right),$$

where x_s^* is the global optimum and $\hat{x}_s$ is the rate vector allocated by these methods.

If each utility function $U_s(\cdot)$ is no longer concave, the proposed methods in the NUM literature can converge to a local optimum or even a saddle point instead of the global optimum (e.g., [4, 7]). This leads to an optimality gap of $O(\mathcal{K} \cdot \max_{s\in[S]} M_s)$, which can be significantly large as the maximum rate M_s increases, representing a substantial utility loss in non-concave landscapes. As traditional decomposition methods for convex optimization can lead to sub-optimal or substantially bad performance under non-concave utility, we are thus well motivated to propose a method to well adapt to unknown and non-concave utility functions. Due to the page limit, we skip some detailed proofs but still introduce sketches in the main text. Please refer to online technical report [11] for details.

3 An ICL Framework for NUM

In this section, we illustrate the design of our transformer-based ICL for NUM. As shown in Figure 2, our ICL framework begins with the example collection stage to gather samples of route-rate pairs from the historical data pool. Then, they are processed to construct a prompt to be embedded and masked in the data processing stage. In the transformer training stage, the output prompt goes through the attention heads to perform self-attention for the transformer's ICL prediction, where attention parameters are trained for the ICL efficiency. Finally, the obtained rate is projected to ensure no violation of the link capacity constraints. We detail our design of each stage in the following.

3.1 Stage I. Example Collection

As shown in Figure 2, our ICL framework begins with example collection. ICL needs data examples of the same pattern for effective prediction. Thus, the network system pre-collects M route-rate data pairs from the historical data pool, where each pair belongs to the same flow set under a fixed users' utility realization.

After overviewing the example collection stage, we give formal definition of the example data below. We define a route-rate pair as (r_s, x_s), where $r_s := (r_s^1, \cdots, r_s^L) \in \mathbb{R}^L$ denotes flow s's route vector and $x_s \in (0, \min_{\ell \in L(s)} c_\ell)$ denotes its source rate allocation. In particular, r_s encodes both flow s's routing structure and the corresponding link-state features, where $r_s^\ell = 0$ if link $\ell \notin L(s)$, and $r_s^\ell = |S(\ell)|$ if $\ell \in L(s)$. This formulation allows the input vector to reflect both the physical connectivity and the instantaneous congestion environment of each flow.

Since the space of route vectors is finite, we summarize all possible flow vectors r_k into a set $\mathcal{R} := \{r_k \in \mathbb{R}^L | k \in [K]\}$, where $\|r_k - r_{k'}\| = \Theta(\Delta)$ with route vector gap $\Delta = \Theta(1)$ for any $k \neq k'$. We suppose r_s is a noisy version of some $r_k \in \mathcal{R}$, satisfying $\|r_s - r_k\| = \mathcal{O}(\Delta)$. For simplicity, we assume $r_s = r_k$ in the later analysis, such that each r_s is randomly sampled from $\mathcal{R}$ with probability $p_k = \Theta(\frac{1}{K})$.

During the example collection stage, a random and independent error ϵ_s can be incurred for each flow s, where we consider $\mathbb{E}[\epsilon_s] = 0$ and $\mathbb{V}[(\epsilon_s)^2] = \sigma^2$ for any flow $s \in [M]$ without much loss of generality. Different from the ICL literature (e.g., [12]), we allow source s's rate x_s to be erroneous (instead of the optimal solution) for each flow r_s as follows:

$$x_s = f(r_s) + \epsilon_s, \tag{2}$$

where $f \in \mathcal{F}$ denotes the optimal mapping between a flow's route r_s and the corresponding source rate x_s under the same user utility combination. The sampled M flow-rate data pairs $\{(r_s, x_s\}_{s \in [M]}$ with the same underlying utility combination will be used to construct a prompt for an efficient transformer training later.

The collection of route-rate data pairs is a well-established practice in data-driven NUM. To train machine learning models, researchers routinely synthesize ground-truth datasets by utilizing exact mathematical solvers to generate optimal rate allocations (e.g., [22]). Furthermore, in environments where utility functions are unknown a priori, frameworks (e.g., Learning-NUM [8]) empirically gather data by dynamically allocating rates and recording the resulting utility feedback to train surrogate models (e.g., [14]).

3.2 Stage II. Data Processing

After obtaining the route-rate data examples, we then perform data processing for the efficiency of ICL prediction later, including prompt construction, embedding and masking as shown in Figure 2.

Prompt construction. We construct a prompt P to include the M route-rate data examples $\{(r_s, x_s)\}_{s \in [M]}$ and a new query flow r_q for the transformer to infer as follows:

$$P = (r_1, x_1; \cdots; r_M, x_M; r_q). \tag{3}$$

Recall that in each prompt P, all the route-rate data pairs $\{(r_s, x_s)\}_{s \in [M]}$ follow the same mapping function yet may incur independent sampling errors as in (2).

Embedding layer. After constructing the prompt, we then follow a natural token embedding in the ICL literature to construct each column $m \in [M]$ as $\begin{pmatrix} r_s \\ x_s \end{pmatrix}$ and the last column as $\begin{pmatrix} r_q \\ 0 \end{pmatrix}$. This embedding helps align the demonstrations and the query in a shared representation space, enabling the model to infer the missing target for the query by leveraging the structural relationships in the preceding examples. Consequently, we obtain the embedding matrix of P as follows:

$$E(P) = \begin{pmatrix} r_1 & r_2 & \cdots & r_M & r_q \\ x_1 & x_2 & \cdots & x_M & 0 \end{pmatrix} \in \mathbb{R}^{(L+1)\times(M+1)}. \tag{4}$$

Masking layer. To improve the ICL prediction efficiency, the system applies a masking operation to the embedding matrix $E(P)$ in (4), producing $\mathcal{M}(E(P))$, which removes the last column to prevent the query input from attending to itself during training. Therefore, we obtain the masked embedding matrix as follows:

$$\mathcal{M}(E(P)) = \begin{pmatrix} r_1 & r_2 & \cdots & r_M \\ x_1 & x_2 & \cdots & x_M \end{pmatrix} \in \mathbb{R}^{(L+1)\times M}.$$

3.3 Stage III. Transformer Training

Similar to the existing ICL literature (e.g., [19], [12]), we consider a simple but fundamental one-layer transformer, which only involves a self-attention layer without any feed-forward network or decoding layer. While simple, this facilitates our later theoretical analysis in Section 5, which demonstrates its effectiveness in optimizing utility maximization and shows enough advantages over the benchmark of Section 3.

As in Figure 2, the transformer performs self-attention on the masked embedding $\mathcal{M}(E(P))$, with formal definition below.

Definition 3.1. A self-attention (SA) layer consists a key matrix $\mathcal{W}^K \in \mathbb{R}^{(L+1)\times(L+1)}$, a query matrix $\mathcal{W}^Q \in \mathbb{R}^{(L+1)\times(L+1)}$, and a value matrix $\mathcal{W}^V \in \mathbb{R}^{(L+1)\times(L+1)}$. Given an embedding E of a prompt P, the self-attention mechanism outputs

$$F_{SA}(E; \mathcal{W}^K, \mathcal{W}^Q, \mathcal{W}^V) = \mathcal{W}^V E \cdot \mathrm{softmax}\Big((\mathcal{W}^K E)^\top \mathcal{W}^Q E\Big), \tag{5}$$

where the softmax function is $\mathrm{softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$ with z_i as the i-th element of vector z.

Define $\theta := (\mathcal{W}^K, \mathcal{W}^Q, \mathcal{W}^V)$. The ICL prediction $\tilde{x}_q$ for the query flow r_q is the last entry of F_{SA} as follows:

$$\tilde{x}_q = \tilde{x}_q(E(P); \theta) = [F_{\mathrm{SA}}(E(P); \theta)]_{(M+1)}. \tag{6}$$

For transformer training to improve the ICL efficiency, we aim to minimize the following squared loss of prediction error:

$$l(\theta) = \frac{1}{2}\mathbb{E}_{f \in \mathcal{F}}[(\tilde{x}_q - x_q^*)^2], \tag{7}$$

where x_q^* is the optimal source rate of flow r_q under a mapping f.

Given $F_{\mathrm{SA}}(E(P^s); \theta)$ in (5), we find that the non-linear softmax function couples the attention weights across all input tokens, making the training loss objective $l(\theta)$ in (7) highly non-convex and interdependent. Thus, it is difficult to explicitly solve the closed-form θ^* for minimizing $l(\theta)$ in (7) with standard techniques in optimization theory. Consequently, we aim to propose a simple algorithm to efficiently train the transformer with a convergence guarantee after a limited number of training steps.

To achieve the convergence guarantee, we employ the gradient descent algorithm to optimize the non-convex and high-dimensional loss functions as follows:

$$\theta^{(t+1)} = \theta^{(t)} - \eta \cdot \nabla_\theta l(\theta^{(t)}), \tag{8}$$

which offers an efficient and scalable way to deal with our highly non-convex objective and is widely adopted in the machine learning literature (e.g., [13], [21], [40]).

Although the gradient descent approach in (8) provides an efficient way to train the transformer parameters, it remains unclear how the parameter θ evolves during the training step, how long the training procedure takes at most, and whether the convergence of the final parameter θ^* can theoretically be guaranteed. Later in Sections 5 and 6, we make a comprehensive theoretical analysis to provide convergence guarantee to the global optimum.

3.4 Stage IV. Projection

As each obtained ICL prediction $\tilde{x}_q$ in Section 4.3 may not satisfy the link capacity constraints, which can be infeasible for the NUM problem. Therefore, we need to further project the predicted rates $\tilde{\boldsymbol{x}}_q = (\tilde{x}_q^1, \cdots, \tilde{x}_q^S)$ of the S flows onto the required space as in (1), while maintaining the smallest change as follows:

$$\begin{aligned} \min_{\hat{\boldsymbol{x}}_q = (\hat{x}_q^1, \cdots, \hat{x}_q^S)} \quad & ||\hat{\boldsymbol{x}}_q - \tilde{\boldsymbol{x}}_q||_2^2 \\ \text{subject to} \quad & \sum_{s \in S(\ell)} \hat{x}_q^s \le c_\ell, \quad \ell = 1, \ldots, L. \end{aligned} \tag{9}$$

Note that (9) is a convex quadratic program with linear constraints, which corresponds to a Euclidean projection onto a convex polytope. It can be efficiently solved using standard first-order methods (e.g., projected gradient descent or dual decomposition), with polynomial-time complexity in the number of flows and links.

In the following, we conduct a rigorous analysis of our ICL framework for convergence guarantee to the global optimum. To show the efficiency of our ICL framework for any example number case, we first consider a sufficient sample case where the example number M covers all the flow case K in Section 5. Then, we extend to an insufficient sample case where the example number M is less than K in Section 6.

4 Theoretical Analysis of Sufficient Sample Case

In this section, we perform a comprehensive analysis of the convergence of our ICL framework under sufficient samples (i.e., $M > K$). In Section 4.1, we first characterize an explicit formulation of our prediction loss $\mathcal{L}$ in (7) for ease of the convergence analysis of our gradient descent training in (8). In Section 4.2, we show our transformer training's convergence in two phases and prove a small gap of our ICL framework to the global optimum within limited training steps. Based on our transformer training analysis, in Section 4.3, we give a theoretical guarantee of total utility under our ICL framework.

4.1 An Explicit Formulation of Prediction Loss

In this subsection, we aim to give an explicit formulation of the prediction loss in (7) for ease of convergence analysis later. First, following the ICL literature (e.g., [19]), we consider a practical class of non-degenerate $\mathcal{L}$-Lipschitz continuous mapping functions between a flow's route r_s and its optimal rate allocation x_s as follows:

Assumption 2 (non-degenerate $\mathcal{L}$-Lipschitz). *Each mapping function f between a flow's route r_s and its optimal rate x_s is $\mathcal{L}$-Lipschitz continuous, i.e.,*

$$|f(r) - f(r')| \le \mathcal{L} \cdot \|r - r'\|_2, \ \ \forall \|r - r'\|_2 = \Theta(\delta_0), \tag{10}$$

where $\mathcal{L} > 0$ and $\delta_0 = O(1)$. Moreover, for every flow vector $r_k \in \mathcal{R}$, there exists some other flow's route $r_{k'} \in \mathcal{R}$ with $k' \neq k$, such that

$$|f(r_k) - f(r_{k'})| = \Theta(\mathcal{L}) \cdot \|r_k - r_{k'}\|_2. \tag{11}$$

For notational convenience, we write $f(r) \in \mathcal{F}$ if $f(r)$ satisfies Assumption 2. The global $\mathcal{L}$-Lipschitz condition in (10) ensures that the mapping f, relating a flow's route vector to its optimal rate, has bounded sensitivity to variations in the network state. This assumption is reasonable because rate updates driven by congestion feedback evolve smoothly over time, preventing instability and oscillations. Such regularity is standard in control-theoretic analysis of flow control and guarantees well-posedness and convergence of the underlying optimization dynamics.

The additional non-degeneracy condition in (11) ensures that the mapping function f preserves sufficient separation between distinct flow states, preventing different network conditions from being mapped to nearly identical source rates. Further, this condition guarantees distinguishability among flows with different link capacities, which is essential for maintaining unique optimal rate allocations and ensuring the stability of the equilibrium solution. There are many typical functions satisfying non-degenerate $\mathcal{L}$-Lipschitz in Assumption 2, such as non-constant linear, cosine, and ReLU functions. In Section 7, we relax Assumption 2 in experiments to verify the near-optimal utility under our ICL framework.

Before introducing the detailed convergence analysis, we first simplify the transformer parameter for ease of decomposition. Note that the key, query and value matrices in (5) can be very involved for a clear analysis. For the efficiency of transformer training and ICL prediciton, we normalize the value matrix $\mathcal{W}^V$ to represent equal contribution from each route-rate pair in a prompt. Further, we consolidate the query and key matrices into one matrix as $\mathcal{W}^{KQ} \in \mathbb{R}^{L \times L}$ in the following forms:

$$\mathcal{W}^V = \begin{pmatrix} 0_{L\times L} & 0_L \\ 0_L^\top & 1 \end{pmatrix}, \quad \mathcal{W}^{KQ} = \begin{pmatrix} Q & 0_L \\ 0_L^\top & 0 \end{pmatrix}. \tag{12}$$

Note that the consolidation operation on the $\mathcal{W}^{KQ}$ matrix does not change the softmax input in F_{SA}. For ease of exposition, we use the notation $\theta = (1, Q)$ to represent all the transformer parameters. Next, we are ready to give the self-attention mechanism in the parameter θ as follows:

$$F_{\text{SA}}(E(P); \theta) = \mathcal{M}(E^W(P)) \cdot \text{softmax}\left(\mathcal{M}(E^x(P))^\top Q E^x(P)\right), \tag{13}$$

where $E^x(P)$ and $E^W(P)$ denote the first L rows and the last row of $E(P)$, respectively. The ICL prediction for the query flow r_q is the last entry of F_{SA} as shown in (6).

To derive the optimal ICL prediction for the query flow case, we need to build the connection between it and any other flow example in the prompt. To achieve this, we first define attention scores to measure how much the new query flow case r_q of P in (4)

attends or relates to other input flow's route vectors $\{r_s\}_{s\in[M]}$ in transformer training as in Figure 2.

Definition 4.1. Given a prompt P in (3) and its embedding $E(P)$ in (4), we define the attention score at time t for self-attention mechanism F_{SA} in (13) with parameter $\theta^{(t)}$ as below.

a) Given $s \in [M]$, the attention score for the s-th flow input r_s is

$$\begin{aligned}\mathrm{attn}_s^{(t)}(\theta^{(t)}; E(P)) &:= \left[\mathrm{softmax}\left(\mathcal{M}(E^x(P))^\top Q E(P)\right)\right]_s \\ &= \frac{\exp\left(\left(E_s^x(P)\right)^\top Q\left(\theta^{(t)}\right) E_{M+1}^x(P)\right)}{\sum_{j\in[M]} \exp\left(\left(E_j^x(P)\right)^\top Q\left(\theta^{(t)}\right) E_{M+1}^x(P)\right)}.\end{aligned}$$

b) For $k \in [K]$, define $\mathcal{R}_k(P) \subset [M]$ as the index set of flow input such that $r_s = r_k$ for $s \in \mathcal{R}_k(P)$. Then the attention score for the k-th flow case is given by

$$\mathrm{Attn}_k^{(t)}(\theta^{(t)}; E(P)) := \sum_{s\in\mathcal{R}_k(P)} \mathrm{attn}_s^{(t)}(\theta^{(t)}; E(P)). \tag{14}$$

For simplicity, we represent $\mathrm{attn}_s^{(t)}(\theta^{(t)}; E(P))$ as $\mathrm{attn}_s^{(t)}$ and $\mathrm{Attn}_k^{(t)}(\theta^{(t)}; E(P))$ as $\mathrm{Attn}_k^{(t)}$, respectively. We further rewrite $\mathcal{R}_k(P)$ as $\mathcal{R}_k$. Then, the ICL prediction $\tilde{x}_q$ in (6) for the current query flow r_q can be rewritten as

$$\tilde{x}_q = \sum_{s\in[M]} \mathrm{attn}_s^{(t)} x_s = \sum_{k\in[K]} \mathrm{Attn}_k^{(t)} f(r_k) + \sum_{s\in[M]} \mathrm{attn}_s^{(t)} \epsilon_s. \tag{15}$$

Intuitively, the ICL prediction $\tilde{x}_q$ in (15) is determined by aggregating the candidate source rates $\{x_s\}_{s\in[M]}$ according to their attention scores. The attention mechanism needs to assign higher weights to those flow cases closer to the current query flow, pushing the prediction toward the optimal rate allocation.

Building on the expression in (15), we next show how the attention scores defined in (14) simplifies the prediction loss defined in (7) in the following lemma.

Lemma 4.2. *Given constants $\mathcal{L}, \Delta > 0$, for any $t \in [T]$, the prediction loss in* (7) *can be expressed as:*

$$\begin{aligned} l(\theta) = \sum_{k=1}^{K} \mathbb{E}\Big[\mathbf{1}\{r_q = r_k\}\Big(\Big(1 - Attn_k^{(t)}\Big)^2 \Theta(\mathcal{L}^2\Delta^2) \\ + \sum_{k\in[K]} (Attn_k^{(t)})^2 \Theta(\sigma^2)\Big)\Big], \end{aligned} \tag{16}$$

where $\mathbf{1}\{r_q = r_k\}$ is the indicator function that equals 1 if $r_q = r_k$ and 0 otherwise.

Lemma 4.2 reveals that the prediction loss $l(\theta)$ depends on the function Lipschitz constant $\mathcal{L}$, the route feature gap Δ, and the attention score $\mathrm{Attn}_k^{(t)}$ associated with the true feature of the query flow (i.e., $r_q = r_k$). Based on the above Lemmas 4.2, we then analyze the convergence of loss $l(\theta)$ in the following.

4.2 Analysis of Convergence of Transformer Training and Testing

The convergence of transformer training can be conceptually divided into two phases. In Phase I, the ICL prediction $\tilde{x}_q$ of the source rate is still far from the global optimum $f(r_q)$, resulting in large prediction errors and rapid parameter updates that drive the model toward the correct solution. During this phase, the training dynamics are dominated by coarse error correction, while finer alignment across different flows evolves more slowly. We formally characterize this training phase in the following.

Proposition 4.3. *In the first training phase $t \in \{1, \cdots, T_1\}$ with $T_1 = \Theta(\frac{K\log(K)}{\eta(\mathcal{L}^2\Delta^2+\sigma^2)})$, we have the attention score $Attn_k^{(T_1+1)} = \Omega(\frac{1}{1+\delta_0})$ in the end.*

According to Proposition 4.3, Training Phase I ends once the ICL prediction $\tilde{x}_q$ becomes sufficiently close to the optimal source rate $f(r_q)$, so that the dominant prediction error is effectively mitigated. Intuitively, Phase I is characterized by large discrepancies between the predicted and optimal rates, during which the model rapidly corrects coarse errors and distinguishes different flow inputs. A larger Lipschitz constant $\mathcal{L}$ implies that small variations in flow inputs lead to larger residual errors, thereby accelerating the correction process. Similarly, a larger flow gap Δ increases the separation between $f(r_k)$ and $f(r_{k'})$, making distinct flow cases easier to differentiate. As a result, the duration T_1 of Training Phase I decreases as either $\mathcal{L}$ or Δ increases.

At the end of Training Phase I with $t = T_1$, the prediction loss in (7) can still be large instead of converging to a small value. Therefore, we need further training steps to steadily improve the attention score $\mathrm{Attn}_k^{(t)}$ from $t = T_1 + 1$. We now formalize this behavior in the second training phase in the following.

Proposition 4.4. *In the second training phase $t \in \{T_1+1, \cdots, T^*\}$ with $T^* = \Theta(\frac{K\log(K\epsilon^{-1})}{\eta(\delta_0^2\mathcal{L}^2\Delta^2+\sigma^2)})$, we obtain $Attn_k^{(T^*)} = \Omega(\frac{1}{1+\delta_0\epsilon})$ in the end.*

Since most of the prediction mismatch has been eliminated in Training Phase I, the transformer in Training Phase II primarily focuses on reinforcing the correct associations between flow inputs and the target prediction. During this phase, the influence of the desired flow case r_k on the prediction increases steadily through repeated refinement, while the impact of other irrelevant flow cases is progressively suppressed. As training proceeds, the model increasingly concentrates on the optimal source rate. By the end of Training Phase II, the resulting attention score $\mathrm{Attn}_k^{(T^*)}$ satisfies $1 - \mathrm{Attn}_k^{(T^*)} = O\left(\frac{\epsilon\delta}{1+\epsilon\delta}\right) = O(\epsilon)$, indicating that the transformer has effectively achieved full attention on the optimal source rate.

Based on the convergence analysis in attention score $\mathrm{Attn}_k^{(t)}$ in Propositions 4.3 and 4.4, we are now ready to establish the convergence guarantee for our transformer training in the following.

Theorem 4.5. *If the number of input examples M in a prompt P and the maximum flow number K satisfy $M \geq poly(K)$, and the gap parameter $\mathcal{L}$ of mapping functions satisfies $\mathcal{L} = O(\frac{1}{\Delta})$, then for any $\epsilon \in (0, 1)$, applying gradient descent to train the loss function $l(\theta)$ ensures convergence. Specifically, with at most $T^* = \Theta(\frac{K\log(K\epsilon^{-1})}{\eta(\delta_0^2\mathcal{L}^2\Delta^2+\sigma^2)})$ iterations, the prediction loss satisfies $l\left(\theta^{(T^*)}\right) = O(\epsilon^2) + \Theta(\sigma^2)$.*

Theorem 4.5 states that with enough data examples M, our transformer training ensures convergence of the loss function $l(\theta)$ in (7) to a small gap within a limited number T^* of training steps, allowing the transformer to output a near-optimal $\tilde{x}_q$ for r_q. However,

as the maximum flow number K increases, each prompt needs to include more data examples to train, which delays the convergence. Conversely, the convergence time T^* in Theorem 4.5 decreases with both the function gap parameter $\mathcal{L}$ and flow vector gap Δ defined in Assumption 1 since a larger $\mathcal{L}$ or Δ results in a greater gradient, which accelerates the loss function convergence. According to Theorem 4.5, we give a complexity analysis of our ICL approach.

Corollary 4.6. *The overall training complexity of our L-dimensional transformer-based ICL framework is*

$$O\left(M^2 L \cdot \frac{K \log\left(K\epsilon^{-1}\right)}{\eta\left(\delta_0^2\, \mathcal{L}^2\, \Delta^2 + \sigma^2\right)}\right).$$

As the obtained ICL prediction $\tilde{x}_q$ may not satisfy the link capacity constraints, which can be infeasible for the NUM problem. Therefore, we need to further project the predicted rate $\tilde{x}_q$ onto the required space as in (9). We have the same bound on the prediction loss with the final prediction $\hat{x}_q$.

Theorem 4.7. *After projecting the ICL prediction $\tilde{x}_q$ to satisfy the link capacity constraints, the final prediction $\hat{x}_q$ maintains the same order of prediction loss as follows:*

$$\mathbb{E}_{f\in\mathcal{F}}[(\hat{x}_q - x_q^*)^2] = O(\epsilon^2) + \Theta(\sigma^2).$$

Building on the convergence established in Theorems 4.5 and 4.7, we now show that our ICL-enhanced approach can infer the optimal source rate for utility combinations not encountered during training, well adapting to unknown utility combinations under dynamic network environments in practice.

Proposition 4.8. *After the prediction loss $l(\theta)$ in converges to $l(\theta) = O(\epsilon^2) + \Theta(\sigma^2)$, for any unseen mapping function (in training) $f^u \in \mathcal{F}$ under an utility combination u, if query flow satisfies $r_q = r_k$, we have $\mathbb{E}(\hat{x}_q - x_q^*)^2 = O(\epsilon^2) + \Theta(\sigma^2)$.*

Intuitively, as the attention score Attn_k approaches 1, the attention mechanism pays attention to prompt examples that share the exact flow case r_k. Consequently, the predicted rate $\tilde{x}_q$ from (15) closely approximates the global optimum x_q, resulting in highly accurate predictions as the projected $\hat{x}_q$ shares the same bound with $\tilde{x}_q$. Crucially, this holds even if the mapping function f^u under an unknown utility combination u was not part of the training set. This capability enables in-context learning at test time, allowing the converged model to generate correct predictions under dynamic network environments without fine-tuning.

Corollary 4.9. *After training the L-dimensional transformer, the inference delay for any new prompt with M tokens is $O(M^2 L)$.*

4.3 Theoretical Guarantee on Utility Loss

To further check our ICL framework's performance on the utilities U_s in (1), we define the average utility loss ΔU as the expected difference between the total utility U^* under global optimal rates $\{x_s\}_{s\in[S]}$ and $\hat{U}$ of the final predictions $\{\hat{x}_s\}_{s\in[S]}$:

$$\Delta U := \frac{1}{S}(U^* - \hat{U}) = \frac{1}{S}\mathbb{E}_{f\in\mathcal{F}}\Big[\sum_{s\in[S]} U_s(x_s) - \sum_{s\in[S]} U_s(\hat{x}_s)\Big]. \quad (17)$$

Recall that we only assume each utility U to be $\mathcal{K}$-Lipschitz continuous in Assumption 2. Together with Theorem 4.7 and Proposition 4.8, we are now ready to bound the utility loss ΔU in (17).

Theorem 4.10. *The utility loss ΔU in (17) of our ICL framework from the global optimum is upper bounded as follows:*

$$\Delta U \le O(\mathcal{K}\sqrt{\epsilon^2 + \sigma^2}).$$

Compared with the benchmark's sub-optimal or arbitrarily bad utility in Remark of Section 2.2, Theorem 4.10 guarantees a limited utility loss for our ICL framework under varying or unknown utility functions, well adaptive to dynamic network environments.

5 Extension to Insufficient Sample Case

Recall that in Section 5, we assume that data examples are sufficient to cover the flow space as $M = \text{poly}(K)$. However, it can be hard to collect sufficient flow data in practice, Inspired by [36], in this section, we extend our ICL framework to insufficient example inputs $M < K$ to further demonstrate how to utilize our ICL framework and prove its convergence to the global optimum.

5.1 Our ICL Framework Adaption for Insufficient Data Examples

As shown in Figure 2, our ICL framework still contains the same stages under insufficient data examples. We highlight the difference in the following.

ICL data examples. With insufficient data samples $M < K$, the query flow does not appear in the prompt. Thus, it is more challenging to predict the optimal rate allocation of the query flow using other route-rate examples. We pick one flow route r_q where $q \in \{M+1, \ldots, K\}$ from the $K - M$ unseen flows to predict its optimal rate allocation, with M insufficient route-rate pairs pre-collected as ICL examples. The flow set $\mathcal{R} := \{r_1, \cdots, r_K\} \subset \mathbb{R}^L$ satisfies $||r_k||_2 \le \bar{r}$ for any $k \in [K]$.

Multi-head attention. We adopt a one-layer transformer with H-head softmax attention, still not involving any feed-forward network or decoding layer. Based on the embedding E in (4), the output of each transformer head is

$$\text{head}_h(E) := \mathcal{W}_h^V \mathcal{M}(E) \cdot \text{softmax}\Big((\mathcal{M}(E))^\top (\mathcal{W}_h^K)^\top \mathcal{W}_h^Q E^x\Big) \quad (18)$$

for $h \in [H]$. The attention map of the transformer $\mathcal{T}(E)$ is

$$\mathcal{T}(E) := \mathcal{W}^O(\text{head}_1(E) \cdots \text{head}_H(E))^\top. \quad (19)$$

For the efficiency of our transformer training, we write the parameter matrices as

$$\mathcal{W}^O := (\mathbf{I}, \cdots, \mathbf{I}) \in \mathbb{R}^{1\times H}, \quad \mathcal{W}_h^V := (0, V_h) \in \mathbb{R}^{1\times(L+1)}, \quad (20)$$

$$(\mathcal{W}_h^K)^\top \mathcal{W}_h^Q = \begin{pmatrix} Q_h & 0 \\ 0 & 0 \end{pmatrix} \in \mathbb{R}^{(L+1)\times(L+1)}, \quad \forall h \in [H]. \quad (21)$$

ICL prediction. The transformer predicts the source rate for the query flow r_q. The ICL prediction $\hat{x}_q$ for flow q is

$$\hat{x}_q = \left\langle x, \sum_{h=1}^{H} V_h \,\text{softmax}\big(R^\top Q_h r_q\big)\right\rangle.$$

The prediction loss is given as

$$l(\theta) := \frac{1}{2}\mathbb{E}\left[\left(\hat{x}_q - x_q^*\right)^2\right].$$

Similar to the case of sufficient ICL examples, we conduct a simple gradient descent for the transformer training as follows:

$$\forall h \in [H]: \quad Q_h^{(t)} = Q_h^{(t-1)} - \eta_Q \nabla_{Q_h} l\big(\theta^{(t-1)}\big), \tag{22}$$

$$V_h^{(t)} = V_h^{(t-1)} - \eta_w \nabla_{V_h} l\big(\theta^{(t-1)}\big), \tag{23}$$

where $\theta^{(t)} = \big\{Q_h^{(t)}, V_h^{(t)}\big\}_{h=1}^{H}$ is the parameter at the t-th training step. In the following, we prove that the transformer training asymptotically converges to the global optimum with a small gap.

5.2 Theoretical Guarantee

We relax Assumption 2 on Lipschitz continuous mapping function in Section 5 and generalize to the following functions for our analysis later.

Assumption 3. *A mapping function $f \in \mathcal{F}$ between a flow's route r and its optimal rate allocation x^* is continuous and can be approximated by a linear span of fixed base functions $\{g_j(\cdot)\}_{j=1}^{N}$:*

$$x^* = f(r) = \sum_{j=1}^{N} \lambda_j\, g_j(r) = \lambda^\top g(r),$$

where $g(\cdot) := (g_1(\cdot), \cdots, g_N(\cdot))^\top$ is the vector of base mapping functions satisfying $||g(\cdot)||_2 \le \bar{x}$ and $\lambda(u) := (\lambda_1, \cdots, \lambda_N)^\top$ is the utility-dependent linear-span vector with $\mathbb{E}[\lambda_j] = 0$ and $\mathbb{V}[\lambda_j^2] = 1$ for all $j \in [N]$. Further, the noisy rate of a flow's route input r_s for any ICL example $s \in [M]$ is given as

$$x_s = \lambda^\top (g(r_s) + \epsilon_s),$$

where each error $\epsilon_{s,i}$ is i.i.d. with $\mathbb{E}[\epsilon_{s,i}] = 0$ and $\mathbb{V}[\epsilon_{s,i}^2] = \sigma^2$.

Note that Assumption 3, followed by Stone–Weierstrass theorem (e.g., [5]), is commonly used in the ML literature (e.g., [29]). It coincides with our NUM problem setting where each flow's rate depends on the collective influence of the other flows along its routing path. In practice, the end-to-end rate of a flow is determined by how each traversed link constrains or enables transmission. This assumption represents the optimal rate as a function of fixed nonlinear route features, enabling flexible approximation of continuous rate–routing relationships.

Before we give the convergence guarantee, let us introduce a few notations for ease of further analysis. We denote

$$Z := \big(g(r_1) \cdots g(r_M)\big) \in \mathbb{R}^{N\times M}, \quad \bar{Z} := \big(Z^\top Z + N\sigma^2 I_M\big)^{1/2} \in \mathbb{R}^{M\times M},$$

where we have $\max_{i\in[M]} \|\bar{z}_i\|_2 \le \sqrt{\bar{x}^2 + N\sigma^2}$ according to the definition of $g(\cdot)$, $\bar{z}_i$ is the i-th column vector of $\bar{Z}$ for $i \in [M]$. We further define $D_q^{(t)}$ ($t \in \mathbb{N}_+$) and $B_q^{(t)}$ as follows:

$$D_q^{(t)} := \mathrm{softmax}\big(R^\top Q_1^{(t)} r_q, \cdots, R^\top Q_H^{(t)} r_q\big) \in \mathbb{R}^{M\times H},$$
$$B_q^{(t)} = \bar{Z} D_q^{(t)} \in \mathbb{R}^{M\times H}. \tag{24}$$

We further define

$$\zeta_0 := \lambda_{\min}\big(B_q^{(0)} B_q^{(0)\top}\big) > 0,$$

and $l(\theta^{(0)})$ is the initial prediction loss. Let γ be any positive constant that satisfies

$$\gamma \;\ge\; \zeta_0^{-5/4} \left(\frac{128\sqrt{2}}{\sqrt{2}-1} \|\bar{Z}\|_2^2 \sqrt{H} \sqrt{\bar{x}^2 + N\sigma^2} \sqrt{l(\theta^{(0)})}\, \bar{r}^2 \right)^{1/2}.$$

Given the above notations, we now give the convergence guarantee of our gradient descent training in the following.

Theorem 5.1. *By setting the learning rates as $\eta_Q \le \frac{1}{\mathcal{L}}$ and $\eta_w = \gamma^2 \eta_Q$, where*

$$\mathcal{L} = \Theta\Big(\bar{r}^2 H^{\frac{3}{4}} \left\|\bar{Z}\right\|_2^5 l(\theta^{(0)}) (\bar{x}^2 + N\sigma^2)^{\frac{1}{4}} \zeta_0^{-\frac{9}{4}} \Big),$$

we have the utility loss from the global optimum satisfies

$$\lim_{t\to\infty} \Delta U^{(t)} \le O(\mathcal{K}\sqrt{N\sigma^2 + \bar{x}^2}).$$

We then check if our transformer can still predict well for unseen utility parameters not encountered during the transformer training. For notational simplicity, let $z_q := g(r_q) \in \mathbb{R}^N$ denote the representation of the query flow. We have the following.

Theorem 5.2. *Let $\hat{\lambda}$ be the solution to the ridge regression problem:*

$$\hat{\lambda} := \arg\min_{\lambda} \left\{ \frac{1}{2M} \sum_{s=1}^{M} \big(x_s - \lambda^\top g(r_s)\big)^2 + \frac{N\sigma^2}{2M} \|\lambda\|_2^2 \right\}.$$

For any $\varepsilon > 0$ and $\delta \in (0,1)$, if the training-step number T satisfies

$$T \ge \Omega\Big(\log\big(\tfrac{\log(1/\delta)}{\varepsilon}\big) \Big),$$

then with probability at least $1-\delta$, the utility loss from the ridge regression optimum satisfies

$$U(x_q^\star) - U(\hat{x}_q) \le O(\mathcal{K}\varepsilon), x_q^\star := \hat{\lambda}^\top z_q.$$

Theorem 5.2 shows that the optimal rate mapping can be reliably recovered from noisy route–rate observations using a regularized least-squares estimator. Ridge regularization balances estimation accuracy and noise sensitivity, ensuring stable recovery of the underlying mapping coefficients. With a logarithmic number of training steps, the learned mapping achieves a small deviation from the optimal utility, implying that limited training suffices to obtain near-optimal rate allocations with high probability.

6 Experiments

In this section, we relax the assumptions in Sections 5-6 to run experiments on practical user utility functions for comparing our ICL framework against state-of-the-art approaches. In Section 6.1, we introduce the experimental setups on transformer training. In Section 6.2, we give experimental results in the testing stage.

6.1 Experimental Setups

As shown in Figure 1, we consider a real-world Abilene network (e.g., [14], [8]) with S=10 flows and L=14 links. We normalize each link capacity as c_ℓ=1 and allow each flow's non-concave utility function to be sigmoid as follows (e.g., [10], [7]):

$$U_s(x_s) = \frac{1}{1 + e^{-a(x_s - b_s)}}, x_s \in [0.1, 1], s \in [10],$$

where a=10 is the global elasticity parameter and b_s denotes the rate level at which flow s becomes satisfied. Without loss of generality, we consider flows 1-3 have interactive traffic (e.g., real-time voice calls) with $b_s \in [0.15, 0.4]$, flows 4-7 have video traffic (e.g., video streaming) with $b_s \in [0.4, 0.7]$ and flows 8-10 have bulk traffic (e.g., file transfer or data backup) with $b_s \in [0.7, 0.9]$.

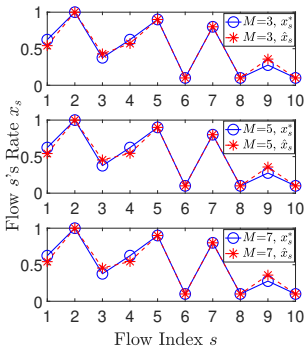

Figure 3: ICL prediction for each flow vs example number $M \in \{3, 5, 7\}$, respectively. Here we fix the distribution shift $\Delta b = 0.3$ in the test.

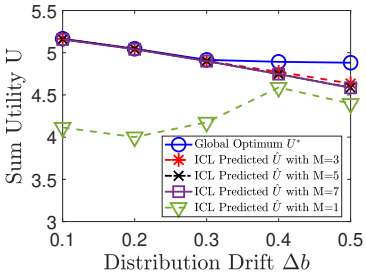

Figure 4: Sum utility U versus distribution drift Δb under example number $M \in \{1, 3, 5, 7\}$, respectively.

To formulate the training prompt, we consider the following sigmoid parameters for the 10 follows:

$$b^{(0)} = [0.18, 0.27, 0.36, 0.43, 0.50, 0.58, 0.66, 0.74, 0.82, 0.89].$$

We use nonlinear constrained optimization via SCIP, a solver for mixed-integer and nonlinear programs, to approximate the optimal rate allocation under training data.

To evaluate our ICL framework's robustness under varying utility functions, we define distribution shift as follows:

$$\Delta b := ||b(i) - b(0)||_2,$$

where $b(i)$ denotes the underlying sigmoid parameter vector in testing i. We choose $\Delta b \in \{0.1, 0.2, 0.3, 0.4, 0.5\}$ for testing.

6.2 Experimental Results in the Testing Stage

Figure 3 shows the output of our ICL framework of each flow versus example number $M \in \{3, 5, 7\}$, where we fix the distribution drift $\Delta b = 0.3$ in the test. It indicates that our ICL framework can approximate optimal rate allocation well for most flows under each example number, consistent with Theorem 4.7 in Section 5.2.

Figure 4 shows the sum utility U versus distribution drift Δb under example number $M \in \{1, 3, 5, 7\}$, respectively. Perhaps surprising, it indicates that the ICL predicted sum utility may not increase with example number in the prompt. As distribution drift increases, testing data becomes more different than training data.

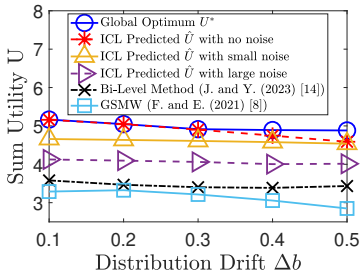

Figure 5: Sum utility U of our ICL framework with different noise levels and state-of-the-art approaches versus distribution drift Δb, respectively. Here we consider random small noise $|\epsilon_s| \leq 0.60$ and random large noise $|\epsilon_s| \leq 1.20$, $s \in [S]$.

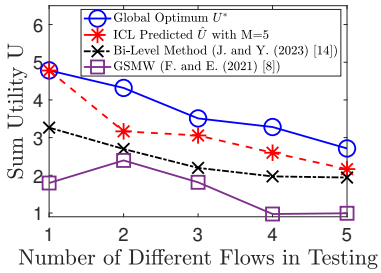

Figure 6: Sum utility U of our ICL framework and state-of-the-art approaches versus flow route change, respectively. In testing, flow routes differ from training as follows: flow 2 changes link $2 \to 4$ to $1 \to 5 \to 6$ in all test sets; flow 7 changes link $12 \to 14$ to $10 \to 8 \to 9 \to 11 \to 13$ from Test Set 2 onward; flow 8 changes link $13 \to 14$ to $11 \to 9 \to 8 \to 10 \to 12$ from Test Set 3 onward; flow 9 changes link $11 \to 13$ to $9 \to 8 \to 10 \to 12 \to 14$ from Test Set 4 onward; and flow 4 changes link $6 \to 9 \to 11$ to $7 \to 10 \to 12 \to 14 \to 13$ in Test Set 5.

Thus, increasing the example number may not be helpful for utility maximization in practice.

Figure 5 shows the sum utility U of our ICL framework with different noise levels and state-of-the-art approaches versus distribution drift Δb, respectively. It indicates that our ICL framework achieves near-optimal sum utility with no noise on each example's rate. As the noise increases, the sum utility of our ICL framework degrades, consistent with Theorem 4.10. Nonetheless, it still outperforms the state-of-the-art approaches even with a large noise.

Figure 6 compares the sum utility U of our ICL framework with state-of-the-art baselines under changing flow routes in testing. While baseline performance degrades as routes deviate from training, our ICL framework consistently achieves the highest utility, indicating robustness to routing variations. This result is consistent with the bounded utility loss guaranteed by Theorem 5.2.

7 Conclusion

In this work, we propose a novel transformer-based ICL framework for NUM under unknown and non-concave utilities. We propose

an efficient algorithm for transformer training to guarantee convergence to the global optimum within limited training steps. As it can be hard to collect sufficient flow data in practice, we extend our ICL framework to insufficient example inputs, still guaranteeing a small gap to the global optimum. Experiments further demonstrate our method's good adaptiveness to varying utility functions, as well as great utility improvement over state-of-the-art solutions on NUM with unknown and non-concave utility functions.

References

[1] Dimitri P. Bertsekas. 1999. *Nonlinear Programming* (2nd ed.). Athena Scientific, Belmont, MA.

[2] Gordon Owusu Boateng, Hani Sami, Ahmed Alagha, Hanae Elmekki, Ahmad Hammoud, Rabeb Mizouni, Azzam Mourad, Hadi Otrok, Jamal Bentahar, Sami Muhaidat, et al. 2025. A survey on large language models for communication, network, and service management: Application insights, challenges, and future directions. *IEEE Communications Surveys & Tutorials* (2025).

[3] Mung Chiang. 2005. Balancing Transport and Physical Layers in Wireless Multihop Networks: Jointly Optimal Congestion Control and Power Control. *IEEE Journal on Selected Areas in Communications* 23, 1 (2005), 104–116. doi:10.1109/JSAC.2004.837332

[4] Mung Chiang, Steven H. Low, A. Robert Calderbank, and John C. Doyle. 2007. Layering as Optimization Decomposition: A Mathematical Theory of Network Architectures. *Proc. IEEE* 95, 1 (2007), 255–312. doi:10.1109/JPROC.2006.887133

[5] Louis De Branges. 1959. The stone-weierstrass theorem. *Proc. Amer. Math. Soc.* 10, 5 (1959), 822–824.

[6] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Jingyuan Ma, Rui Li, Heming Xia, Jingjing Xu, Zhiyong Wu, Baobao Chang, et al. 2024. A Survey on In-context Learning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 1107–1128.

[7] Maryam Fazel and Mung Chiang. 2005. Network utility maximization with nonconcave utilities using sum-of-squares method. In *Proceedings of the 44th IEEE Conference on Decision and Control*. IEEE, 1867–1874.

[8] Xinzhe Fu and Eytan Modiano. 2021. Learning-NUM: Network utility maximization with unknown utility functions and queueing delay. In *Proceedings of the Twenty-second International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing*. 21–30.

[9] Leonidas Georgiadis, Michael J Neely, and Leandros Tassiulas. 2006. *Resource allocation and cross-layer control in wireless networks*. Now Publishers Inc.

[10] Prashanth Hande, Shengyu Zhang, and Mung Chiang. 2007. Distributed rate allocation for inelastic flows. *IEEE/ACM Transactions on Networking* 15, 6 (2007), 1240–1253.

[11] S. Hao and L. Duan. 2026. Transformer-Based In-Context Learning for Unknown and Non-Concave Network Utility Maximization. Techincal Report. https://shuganghao.github.io/files/hao2026ICL.pdf

[12] Shugang Hao, Hongbo Li, and Lingjie Duan. 2025. To Theoretically Understand Transformer-Based In-Context Learning for Optimizing CSMA. In *Proceedings of the Twenty-Sixth International Symposium on Mobile Ad Hoc Networking and Computing (MobiHoc'25)*. ACM, Houston, TX, USA, 1–XX. doi:10.1145/3704413.3764423 arXiv:2508.09146 [cs.LG].

[13] Yu Huang, Yuan Cheng, and Yingbin Liang. 2024. In-context Convergence of Transformers. In *International Conference on Machine Learning*. PMLR, 19660–19722.

[14] Kaiyi Ji and Lei Ying. 2023. Network utility maximization with unknown utility functions: A distributed, data-driven bilevel optimization approach. In *Proceedings of the Twenty-fourth International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing*. 131–140.

[15] Frank P. Kelly, Aman K. Maulloo, and David K. H. Tan. 1998. Rate Control for Communication Networks: Shadow Prices, Proportional Fairness and Stability. *Journal of the Operational Research Society* 49, 3 (1998), 237–252. doi:10.1057/palgrave.jors.2600523

[16] J-W Lee, Ravi R Mazumdar, and Ness B Shroff. 2005. Non-convex optimization and rate control for multi-class services in the Internet. *IEEE/ACM transactions on networking* 13, 4 (2005), 827–840.

[17] Woongsup Lee and Jeonghun Park. 2024. LLM-empowered resource allocation in wireless communications systems. *arXiv preprint arXiv:2408.02944* (2024).

[18] Hongbo Li and Lingjie Duan. 2025. Theory of mixture-of-experts for mobile edge computing. In *IEEE INFOCOM 2025-IEEE Conference on Computer Communications*. IEEE, 1–10.

[19] Hongbo Li, Lingjie Duan, and Yingbin Liang. 2025. Provable In-Context Learning of Nonlinear Regression with Transformers. *arXiv preprint arXiv:2507.20443* (2025).

[20] Hongbo Li, Sen Lin, Lingjie Duan, Yingbin Liang, and Ness Shroff. 2025. Theory on Mixture-of-Experts in Continual Learning. In *The Thirteenth International Conference on Learning Representations*.

[21] Hongkang Li, Meng Wang, Songtao Lu, Xiaodong Cui, and Pin-Yu Chen. 2024. How do nonlinear transformers learn and generalize in in-context learning?. In *Proceedings of the 41st International Conference on Machine Learning*. 28734–28783.

[22] Rui Li, Chaoyun Zhang, Pan Cao, Paul Patras, and John S Thompson. 2018. DELMU: A deep learning approach to maximising the utility of virtualised millimetre-wave backhauls. In *International Conference on Machine Learning for Networking*. Springer, 146–165.

[23] Xin Liu, Honghao Wei, and Lei Ying. 2024. Optimistic joint flow control and link scheduling with unknown utility functions. In *Proceedings of the Twenty-fifth International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing*. 271–280.

[24] Steven H. Low and David E. Lapsley. 1999. Optimization Flow Control, I: Basic Algorithm and Convergence. *IEEE/ACM Transactions on Networking* 7, 6 (1999), 861–874. doi:10.1109/90.811445

[25] Michael J. Neely. 2010. *Stochastic Network Optimization with Application to Communication and Queueing Systems*. Morgan & Claypool.

[26] Michael J. Neely, Eytan Modiano, and Charles E. Rohrs. 2005. Dynamic Power Allocation and Routing for Time-Varying Wireless Networks. *IEEE Journal on Selected Areas in Communications* 23, 1 (2005), 89–103. doi:10.1109/JSAC.2004.837327

[27] Daniel P. Palomar and Mung Chiang. 2006. A Tutorial on Decomposition Methods for Network Utility Maximization. *IEEE Journal on Selected Areas in Communications* 24, 8 (2006), 1439–1451. doi:10.1109/JSAC.2006.879350

[28] Zhiwen Pang, Kang Wei, Long Shi, Zhe Wang, Jun Li, and Feng Shu. 2026. Low-latency Federated LLM Fine-tuning Over Wireless Networks. *arXiv preprint arXiv:2602.01024* (2026).

[29] Philipp Petersen and Felix Voigtlaender. 2018. Optimal approximation of piecewise smooth functions using deep ReLU neural networks. *Neural Networks* 108 (2018), 296–330.

[30] Nurullah Sevim, Mostafa Ibrahim, and Sabit Ekin. 2024. Large language models (LLMs) assisted wireless network deployment in urban settings. In *2024 IEEE 100th Vehicular Technology Conference (VTC2024-Fall)*. IEEE, 1–7.

[31] Zihang Song, Osvaldo Simeone, and Bipin Rajendran. 2024. Neuromorphic In-Context Learning for energy-efficient MIMO symbol detection. In *2024 IEEE 25th International Workshop on Signal Processing Advances in Wireless Communications (SPAWC)*. IEEE, 1–5.

[32] Alexander L. Stolyar. 2009. Maximizing Queueing Network Utility Subject to Stability: Greedy Primal-Dual Algorithm. *Queueing Systems* 63, 1 (2009), 401–424. doi:10.1007/s11134-009-9132-6

[33] Leandros Tassiulas and Anthony Ephremides. 1990. Stability properties of constrained queueing systems and scheduling policies for maximum throughput in multihop radio networks. In *29th IEEE Conference on Decision and Control*. IEEE, 2130–2132.

[34] Leandros Tassiulas and Anthony Ephremides. 1992. Stability Properties of Constrained Queueing Systems and Scheduling Policies for Maximum Throughput in Multihop Radio Networks. *IEEE Trans. Automat. Control* 37, 12 (1992), 1936–1948. doi:10.1109/9.182479

[35] Arun Verma and Manjesh K Hanawal. 2020. Stochastic network utility maximization with unknown utilities: Multi-armed bandits approach. In *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 189–198.

[36] Tong Yang, Yu Huang, Yingbin Liang, and Yuejie Chi. 2024. In-Context Learning with Representations: Contextual Generalization of Trained Transformers. In *Advances in Neural Information Processing Systems 37*. Neural Information Processing Systems Foundation, Inc., 85867–85898. doi:10.52202/079017-2727

[37] Yung Yi and Mung Chiang. 2008. Stochastic network utility maximisation—a tribute to Kelly's paper published in this journal a decade ago. *European Transactions on Telecommunications* 19, 4 (2008), 421–442.

[38] Wei Yu and Raymond Lui. 2006. Dual Methods for Nonconvex Spectrum Optimization of Multicarrier Systems. *IEEE Transactions on Communications* 54, 7 (2006), 1310–1322. doi:10.1109/TCOMM.2006.877371

[39] Han Zhang, Akram Bin Sediq, Ali Afana, and Melike Erol-Kantarci. 2024. Large language models in wireless application design: In-context learning-enhanced automatic network intrusion detection. In *GLOBECOM 2024-2024 IEEE Global Communications Conference*. IEEE, 2479–2484.

[40] Ruiqi Zhang, Spencer Frei, and Peter L. Bartlett. 2024. Trained Transformers Learn Linear Models In-Context. *Journal of Machine Learning Research* 25, 49 (2024), 1–55. https://jmlr.org/papers/v25/23-1042.html

[41] Songge Zhang, Guoliang Cheng, Wen Wu, Xinyu Huang, Lingyang Song, and Xuemin Shen. 2025. Split Fine-Tuning for Large Language Models in Wireless Networks. *IEEE Journal of Selected Topics in Signal Processing* 19, 7 (2025), 1104–1118. doi:10.1109/JSTSP.2025.3581484

[42] Hao Zhou, Chengming Hu, Dun Yuan, Ye Yuan, Di Wu, Xue Liu, and Charlie Zhang. 2024. Large language model (llm)-enabled in-context learning for wireless network optimization: A case study of power control. *arXiv preprint arXiv:2408.00214* (2024).

Appendix: Proofs of Main Results

In the following, we provide detailed proofs for all the theoretical results in Sections 4 and 5. For ease of reference, we first define auxiliary notation used throughout the proofs and then present two supporting lemmas that are key to the convergence analysis of Sections 4.1–4.2.

Auxiliary notation. Following the ICL literature (e.g., [13, 19]), we define the *bilinear attention weights* $A_k^{(t)}$, $B_{k,k'}^{(t)}$ and the associated *gradient projections* $\alpha_k^{(t)}$, $\beta_{k,k'}^{(t)}$ for $k, k' \in [K]$ with $k' \neq k$ and training step $t \geq 0$:

$$A_k^{(t)} := r_k^\top Q^{(t)} r_k, \qquad \alpha_k^{(t)} := -r_k^\top \nabla_{Q^{(t)}} l(Q^{(t)}) r_k, \tag{25}$$

$$B_{k,k'}^{(t)} := r_{k'}^\top Q^{(t)} r_k, \qquad \beta_{k,k'}^{(t)} := -r_{k'}^\top \nabla_{Q^{(t)}} l(Q^{(t)}) r_k. \tag{26}$$

The bilinear attention weights $A_k^{(t)}$ and $B_{k,k'}^{(t)}$ capture how the transformer relates different flow inputs: the self-term $A_k^{(t)}$ quantifies how strongly a flow input r_k influences its own rate prediction, while the cross-term $B_{k,k'}^{(t)}$ reflects how flow $r_{k'}$ affects the prediction for r_k. By the gradient descent update in (8), these evolve as:

$$A_k^{(t+1)} = A_k^{(t)} + \eta\, \alpha_k^{(t)}, \qquad B_{k,k'}^{(t+1)} = B_{k,k'}^{(t)} + \eta\, \beta_{k,k'}^{(t)}. \tag{27}$$

We now present two auxiliary lemmas.

Lemma .1 (Balanced data input). *Suppose that $K^3 = O(M)$. For some constant $\rho \geq \sqrt{\frac{20K^3}{M}}$, define the set of prompt inputs*

$$\mathcal{E}^* := \left\{ P_{1:M}^{(t)} : |\mathcal{R}_k| \in \left[p_k M - \frac{\rho M}{K},\ p_k M + \frac{\rho M}{K} \right],\ \forall\, k \in [K] \right\}.$$

Then $\mathbb{P}(P_{1:M}^{(t)} \in \mathcal{E}^*) \geq 1 - 3\exp\left(-\frac{\rho^2 M}{25K^2}\right)$.

Proof. Since each r_s is independently sampled from $\mathcal{R}$ with probabilities $(p_1, \ldots, p_K)$, the cardinalities $\{|\mathcal{R}_k|\}_{k\in[K]}$ follow a multinomial distribution $\text{Multinomial}(M, p_1, \ldots, p_K)$. Let $\delta = \rho/K$, so that $\delta^2/20 \geq K/M$ by our assumption on ρ. Applying the tail bound for the multinomial distribution (e.g., [19, Lemma 2.2]), we obtain

$$\mathbb{P}\left(\sum_{k=1}^{K} \big| |\mathcal{R}_k| - \mathbb{E}[|\mathcal{R}_k|] \big| > \frac{\rho M}{K} \right) \leq 3\exp\left(-\frac{\rho^2 M}{25K^2}\right).$$

Since $\mathbb{E}[|\mathcal{R}_k|] = p_k M$, the event $\{\big||\mathcal{R}_k| - p_k M\big| > \rho M/K\}$ for any single k is contained in the event $\{\sum_{k=1}^{K} \big||\mathcal{R}_k| - p_k M\big| > \rho M/K\}$. By a union-bound argument:

$$\mathbb{P}\left(\bigcup_{k=1}^{K} \left\{ \big||\mathcal{R}_k| - p_k M\big| > \frac{\rho M}{K} \right\} \right) \leq \mathbb{P}\left(\sum_{k=1}^{K} \big||\mathcal{R}_k| - p_k M\big| > \frac{\rho M}{K} \right) \leq 3\exp\left(-\frac{\rho^2 M}{25K^2}\right).$$

Taking the complement yields $\mathbb{P}(P_{1:M}^{(t)} \in \mathcal{E}^*) \geq 1 - 3\exp(-\rho^2 M/(25K^2))$, as claimed. For ease of notation, we define $u_k := (p_k - \rho/K)$ and $U_k := (p_k + \rho/K)$, which satisfy $u_k = \Theta(1/K)$ and $U_k = \Theta(1/K)$ given $p_k = \Theta(1/K)$ and $\rho = O(1)$. According to Lemma .1, for any $P_{1:M}^{(t)} \in \mathcal{E}^*$, we have $|\mathcal{R}_k| \in [u_k M, U_k M] = \Theta(M/K)$ with probability close to 1. □

Lemma .2 (Gradient characterization). *Suppose $r_q = r_k$. Then for any training step $t \in [T]$, flow cases $k \in [K]$ and $k' \in [K]$ with $k' \neq k$, the gradient projections $\alpha_k^{(t)}$ and $\beta_{k,k'}^{(t)}$ defined in* (25)–(26) *satisfy:*

$$\alpha_k^{(t)} = \mathbb{E}\left[\mathbf{1}\{r_q = r_k\}\, \text{Attn}_k^{(t)} \left(\left(1 - \text{Attn}_k^{(t)}\right)^2 \Theta(\mathcal{L}^2 \Delta^2) + \Theta(\sigma^2) \right) \right], \tag{28}$$

$$|\beta_{k,k'}^{(t)}| = \mathbb{E}\left[\mathbf{1}\{r_q = r_k\}\, \text{Attn}_{k'}^{(t)} \left((1 - \text{Attn}_k^{(t)})(1 - \text{Attn}_{k'}^{(t)}) O(\mathcal{L}^2 \Delta^2) + O(\sigma^2) \right) \right]. \tag{29}$$

Proof. We compute $r_{k'}^\top \nabla_{Q^{(t)}} l\, r_k$ by differentiating the loss $l(\theta)$ with respect to Q and projecting onto the bilinear form $r_{k'} r_k^\top$. From the self-attention mechanism in (13), we note that the gradient enters through the softmax function. Since $E_{M+1}^x = r_q$, the projection $r_{k'}^\top \nabla_Q l\, r_k$ is nonzero only when $r_q = r_k$ (or $r_q = r_{k'}$).

Step 1: General bilinear gradient. By the chain rule applied to $l(\theta) = \frac{1}{2}\mathbb{E}_{f\in\mathcal{F}}[(\tilde{x}_q - x_q^*)^2]$ and the softmax structure in (13):

$$r_{k'}^\top \nabla_{Q^{(t)}} l\, r_k = \mathbb{E}\left[\mathbf{1}\{r_q = r_k\}\, (\tilde{x}_q - f(r_k)) \sum_{i,j\in[M]} \text{attn}_i^{(t)}\, \text{attn}_j^{(t)}\, x_i\, r_{k'}^\top (E_i^x - E_j^x) \right].$$

Step 2: Grouping by flow cases. Since all examples in $\mathcal{R}_m$ share the same route r_m, we group the double sum by flow cases and expand. Following the algebra in [13, 19], we substitute $\tilde{x}_q = \sum_{n\in[K]} \mathrm{Attn}_n^{(t)} f(r_n) + \sum_{s\in[M]} \mathrm{attn}_s^{(t)} \epsilon_s$ and rearrange to obtain:

$$r_{k'}^\top \nabla_{Q^{(t)}} l\, r_k = -\mathbb{E}\Bigg[\mathbf{1}\{r_q = r_k\}\Bigg(\mathrm{Attn}_{k'}^{(t)} \underbrace{\Big(f(r_k) - \sum_{n\in[K]} \mathrm{Attn}_n^{(t)} f(r_n)\Big)}_{=:\mathcal{S}_k} \underbrace{\Big(f(r_{k'}) - \sum_{m\in[K]} \mathrm{Attn}_m^{(t)} f(r_m)\Big)}_{=:\mathcal{S}_{k'}} + \underbrace{\Big(\sum_{s\in[M]} \mathrm{attn}_s^{(t)}\, \epsilon_s\Big)\Big(\sum_{i\in\mathcal{R}_{k'}} \mathrm{attn}_i^{(t)}\, \epsilon_i - \mathrm{Attn}_{k'}^{(t)} \sum_{s\in[M]} \mathrm{attn}_s^{(t)}\, \epsilon_s\Big)}_{=:\mathcal{N}_{k'}}\Bigg)\Bigg], \tag{30}$$

where $\mathcal{S}_k$ and $\mathcal{S}_{k'}$ are the *signal residuals* and $\mathcal{N}_{k'}$ is the *noise cross-product.*

Step 3: Evaluating the noise cross-product $\mathbb{E}[\mathcal{N}_{k'}]$. Since the errors $\{\epsilon_s\}_{s\in[M]}$ are independent with $\mathbb{E}[\epsilon_s] = 0$ and $\mathbb{E}[\epsilon_s^2] = \sigma^2$, cross-terms with $s \neq i$ vanish, giving:

$$\begin{aligned}\mathbb{E}[\mathcal{N}_{k'}] &= \mathbb{E}\Bigg[\sum_{i\in\mathcal{R}_{k'}} (\mathrm{attn}_i^{(t)})^2 \epsilon_i^2 - \mathrm{Attn}_{k'}^{(t)} \sum_{s\in[M]} (\mathrm{attn}_s^{(t)})^2 \epsilon_s^2\Bigg] \\ &= \sigma^2\Bigg(\sum_{i\in\mathcal{R}_{k'}} (\mathrm{attn}_i^{(t)})^2 - \mathrm{Attn}_{k'}^{(t)} \sum_{s\in[M]} (\mathrm{attn}_s^{(t)})^2\Bigg).\end{aligned} \tag{31}$$

Since all examples within class j share the same route, the individual attention scores are uniform within each class: $\mathrm{attn}_s^{(t)} = \mathrm{Attn}_j^{(t)}/|\mathcal{R}_j|$ for $s \in \mathcal{R}_j$. Therefore $\sum_{i\in\mathcal{R}_{k'}} (\mathrm{attn}_i^{(t)})^2 = (\mathrm{Attn}_{k'}^{(t)})^2/|\mathcal{R}_{k'}|$ and $\sum_{s\in[M]} (\mathrm{attn}_s^{(t)})^2 = \sum_{j\in[K]} (\mathrm{Attn}_j^{(t)})^2/|\mathcal{R}_j|$. Substituting into (31):

$$\mathbb{E}[\mathcal{N}_{k'}] = \sigma^2\Bigg(\frac{(\mathrm{Attn}_{k'}^{(t)})^2}{|\mathcal{R}_{k'}|} - \mathrm{Attn}_{k'}^{(t)} \sum_{j\in[K]} \frac{(\mathrm{Attn}_j^{(t)})^2}{|\mathcal{R}_j|}\Bigg) = -\sigma^2\, \mathrm{Attn}_{k'}^{(t)} \sum_{j\neq k'} \frac{(\mathrm{Attn}_j^{(t)})^2}{|\mathcal{R}_j|}. \tag{32}$$

Note that $\mathbb{E}[\mathcal{N}_{k'}] \leq 0$, so the noise cross-product contributes *positively* to the gradient projections $\alpha_k^{(t)}$ and $\beta_{k,k'}^{(t)}$ (after the sign flip $-r_{k'}^\top \nabla_Q l\, r_k$).

Using $|\mathcal{R}_j| = \Theta(M/K)$ from Lemma .1, we simplify:

$$-\mathbb{E}[\mathcal{N}_{k'}] = \sigma^2\, \mathrm{Attn}_{k'}^{(t)} \cdot \Theta\Big(\frac{K}{M}\Big) \sum_{j\neq k'} (\mathrm{Attn}_j^{(t)})^2 = \mathrm{Attn}_{k'}^{(t)} \cdot \Theta(\sigma^2), \tag{33}$$

where the final step absorbs $K/M = O(1)$ (since $M = \mathrm{poly}(K)$) and uses $\sum_{j\neq k'} (\mathrm{Attn}_j^{(t)})^2 \leq 1$.

Case $k' = k$ (self-gradient $\alpha_k^{(t)}$): Setting $k' = k$ in (30) and taking $\alpha_k^{(t)} = -r_k^\top \nabla_Q l\, r_k$:

$$\alpha_k^{(t)} = \mathbb{E}\Big[\mathbf{1}\{r_q = r_k\}\Big(\mathrm{Attn}_k^{(t)}\, \mathcal{S}_k^2 - \mathbb{E}[\mathcal{N}_k]\Big)\Big].$$

Using (32) with $k' = k$ gives $-\mathbb{E}[\mathcal{N}_k] = \sigma^2\, \mathrm{Attn}_k^{(t)} \sum_{j\neq k} \frac{(\mathrm{Attn}_j^{(t)})^2}{|\mathcal{R}_j|}$, so:

$$\alpha_k^{(t)} = \mathbb{E}\Bigg[\mathbf{1}\{r_q = r_k\}\, \mathrm{Attn}_k^{(t)} \Bigg(\mathcal{S}_k^2 + \sigma^2 \sum_{j\neq k} \frac{(\mathrm{Attn}_j^{(t)})^2}{|\mathcal{R}_j|}\Bigg)\Bigg].$$

For the signal term, $\mathcal{S}_k = \sum_{n\neq k} \mathrm{Attn}_n^{(t)} (f(r_k) - f(r_n))$, so by the non-degeneracy condition in Assumption 2 ($|f(r_k) - f(r_n)| = \Theta(\mathcal{L}\Delta)$ for $n \neq k$) and $\sum_{n\neq k} \mathrm{Attn}_n^{(t)} = 1 - \mathrm{Attn}_k^{(t)}$, we have $\mathcal{S}_k^2 = (1 - \mathrm{Attn}_k^{(t)})^2\, \Theta(\mathcal{L}^2\Delta^2)$. Combined with (33):

$$\alpha_k^{(t)} = \mathbb{E}\Big[\mathbf{1}\{r_q = r_k\}\, \mathrm{Attn}_k^{(t)} \Big((1 - \mathrm{Attn}_k^{(t)})^2\, \Theta(\mathcal{L}^2\Delta^2) + \Theta(\sigma^2)\Big)\Big],$$

establishing (28).

Case $k' \neq k$ (cross-gradient $\beta_{k,k'}^{(t)}$): Setting $k' \neq k$ in (30) and taking $\beta_{k,k'}^{(t)} = -r_{k'}^\top \nabla_Q l\, r_k$:

$$\beta_{k,k'}^{(t)} = \mathbb{E}\Big[\mathbf{1}\{r_q = r_k\}\Big(\mathrm{Attn}_{k'}^{(t)}\, \mathcal{S}_k\, \mathcal{S}_{k'} - \mathbb{E}[\mathcal{N}_{k'}]\Big)\Big].$$

Using (32):

$$\beta_{k,k'}^{(t)} = \mathbb{E}\Bigg[\mathbf{1}\{r_q = r_k\}\, \mathrm{Attn}_{k'}^{(t)} \Bigg(\mathcal{S}_k\, \mathcal{S}_{k'} + \sigma^2 \sum_{j\neq k'} \frac{(\mathrm{Attn}_j^{(t)})^2}{|\mathcal{R}_j|}\Bigg)\Bigg].$$

Taking absolute values and applying the triangle inequality with $|f(r_n) - f(r_k)| = \Theta(\mathcal{L}\Delta)$:

$$|\mathcal{S}_k| = \left| \sum_{n \neq k} \mathrm{Attn}_n^{(t)} (f(r_k) - f(r_n)) \right| \leq (1 - \mathrm{Attn}_k^{(t)})\, \Theta(\mathcal{L}\Delta),$$
$$|\mathcal{S}_{k'}| = \left| \sum_{m \neq k'} \mathrm{Attn}_m^{(t)} (f(r_{k'}) - f(r_m)) \right| \leq (1 - \mathrm{Attn}_{k'}^{(t)})\, \Theta(\mathcal{L}\Delta).$$

Using these bounds together with the noise evaluation (33):

$$|\beta_{k,k'}^{(t)}| \leq \mathbb{E}\Big[\mathbf{1}\{r_q = r_k\}\, \mathrm{Attn}_{k'}^{(t)} \Big((1 - \mathrm{Attn}_k^{(t)})(1 - \mathrm{Attn}_{k'}^{(t)})\, O(\mathcal{L}^2\Delta^2) + O(\sigma^2) \Big) \Big],$$

establishing (29). □

A Proof of Lemma 4.2

Proof. Recall from (15) that the ICL prediction can be decomposed as

$$\tilde{x}_q = \sum_{s \in [M]} \mathrm{attn}_s^{(t)}\, x_s = \sum_{k \in [K]} \mathrm{Attn}_k^{(t)}\, f(r_k) + \sum_{s \in [M]} \mathrm{attn}_s^{(t)}\, \epsilon_s.$$

Suppose $r_q = r_k$ for some $k \in [K]$, so the optimal rate is $x_q^* = f(r_k)$. Then the squared prediction error is:

$$\begin{aligned}
(\tilde{x}_q - x_q^*)^2 &= \Bigg(\sum_{n \in [K]} \mathrm{Attn}_n^{(t)}\, f(r_n) - f(r_k) + \sum_{s \in [M]} \mathrm{attn}_s^{(t)}\, \epsilon_s \Bigg)^2 \\
&= \Bigg(\sum_{n \neq k} \mathrm{Attn}_n^{(t)} \big(f(r_n) - f(r_k) \big) \Bigg)^2 + \Bigg(\sum_{s \in [M]} \mathrm{attn}_s^{(t)}\, \epsilon_s \Bigg)^2 \\
&\quad + 2 \Bigg(\sum_{n \neq k} \mathrm{Attn}_n^{(t)} \big(f(r_n) - f(r_k) \big) \Bigg) \Bigg(\sum_{s \in [M]} \mathrm{attn}_s^{(t)}\, \epsilon_s \Bigg),
\end{aligned} \tag{34}$$

where we used $\sum_{n \in [K]} \mathrm{Attn}_n^{(t)} = 1$, so that $\sum_{n \in [K]} \mathrm{Attn}_n^{(t)} f(r_n) - f(r_k) = \sum_{n \neq k} \mathrm{Attn}_n^{(t)} (f(r_n) - f(r_k))$.

Taking expectations over $f \in \mathcal{F}$ and the noise $\{\epsilon_s\}$, the cross-term vanishes since $\mathbb{E}[\epsilon_s] = 0$ and ϵ_s is independent of the route vectors. Therefore:

$$\mathbb{E}[(\tilde{x}_q - x_q^*)^2 \mid r_q = r_k] = \Bigg(\sum_{n \neq k} \mathrm{Attn}_n^{(t)} (f(r_n) - f(r_k)) \Bigg)^2 + \sum_{s \in [M]} (\mathrm{attn}_s^{(t)})^2\, \sigma^2.$$

Bounding the signal term. By the non-degeneracy condition in Assumption 2, for any $n \neq k$, we have $|f(r_n) - f(r_k)| = \Theta(\mathcal{L}) \cdot \|r_n - r_k\|_2 = \Theta(\mathcal{L}\Delta)$. Since $\sum_{n \neq k} \mathrm{Attn}_n^{(t)} = 1 - \mathrm{Attn}_k^{(t)}$:

$$\Bigg(\sum_{n \neq k} \mathrm{Attn}_n^{(t)} (f(r_n) - f(r_k)) \Bigg)^2 = (1 - \mathrm{Attn}_k^{(t)})^2\, \Theta(\mathcal{L}^2\Delta^2).$$

Bounding the noise term. For $s \in \mathcal{R}_m$, the attention scores are identical since all examples in $\mathcal{R}_m$ share the same route r_m. Thus $\mathrm{attn}_s^{(t)} = \mathrm{Attn}_m^{(t)} / |\mathcal{R}_m|$ for all $s \in \mathcal{R}_m$. This gives:

$$\sum_{s \in [M]} (\mathrm{attn}_s^{(t)})^2 = \sum_{m \in [K]} \frac{(\mathrm{Attn}_m^{(t)})^2}{|\mathcal{R}_m|} = \Theta\Big(\frac{K}{M}\Big) \sum_{m \in [K]} (\mathrm{Attn}_m^{(t)})^2,$$

where we used $|\mathcal{R}_m| = \Theta(M/K)$ from Lemma .1. Since $M = \mathrm{poly}(K)$ and the ratio K/M is absorbed into the $\Theta(\cdot)$ notation, the noise contribution becomes $\sum_{m \in [K]} (\mathrm{Attn}_m^{(t)})^2\, \Theta(\sigma^2)$.

Combining over all flow cases. The prediction loss is:

$$\begin{aligned}
l(\theta) &= \frac{1}{2} \mathbb{E}_{f \in \mathcal{F}} \big[(\tilde{x}_q - x_q^*)^2 \big] = \frac{1}{2} \sum_{k=1}^{K} \mathbb{E}\Big[\mathbf{1}\{r_q = r_k\}\, \mathbb{E}\big[(\tilde{x}_q - x_q^*)^2 \mid r_q = r_k \big] \Big] \\
&= \sum_{k=1}^{K} \mathbb{E}\Bigg[\mathbf{1}\{r_q = r_k\} \Bigg(\big(1 - \mathrm{Attn}_k^{(t)}\big)^2 \Theta(\mathcal{L}^2\Delta^2) + \sum_{m \in [K]} (\mathrm{Attn}_m^{(t)})^2\, \Theta(\sigma^2) \Bigg) \Bigg],
\end{aligned}$$

which completes the proof. □

B Proof of Proposition 4.3

Proof. The proof proceeds in three steps: (i) characterizing the gradient magnitudes, (ii) tracking the evolution of the bilinear attention weights, and (iii) bounding the attention score at the end of Training Phase I.

Step 1: Gradient magnitudes in Phase I. At initialization $t = 0$, we have $Q^{(0)} = 0$ (or a small random initialization), so $A_k^{(0)} = B_{k,k'}^{(0)} = 0$ for all k, k'. Consequently, the initial attention scores are uniform: $\mathrm{Attn}_k^{(0)} = |\mathcal{R}_k|/M = \Theta(1/K)$ for each $k \in [K]$, and $1 - \mathrm{Attn}_k^{(0)} = \Theta(1)$.

For $t \in \{1, \dots, T_1\}$ (Phase I), the attention scores remain $\mathrm{Attn}_k^{(t)} = \Theta(1)$ and $1 - \mathrm{Attn}_k^{(t)} = \Theta(1)$ since the bilinear weights have not yet grown large enough to concentrate attention. Applying Lemma .2 with $p_k = \Theta(1/K)$:

$$\alpha_k^{(t)} = p_k \cdot \mathbb{E}\Big[\mathrm{Attn}_k^{(t)}\Big((1 - \mathrm{Attn}_k^{(t)})^2\, \Theta(\mathcal{L}^2\Delta^2) + \Theta(\sigma^2)\Big) \,\Big|\, r_q = r_k\Big] = \Theta\left(\frac{\mathcal{L}^2\Delta^2 + \sigma^2}{K}\right), \tag{35}$$

$$|\beta_{k,k'}^{(t)}| = p_k \cdot \mathbb{E}\Big[\mathrm{Attn}_{k'}^{(t)}\Big((1-\mathrm{Attn}_k^{(t)})(1-\mathrm{Attn}_{k'}^{(t)})\mathcal{O}(\mathcal{L}^2\Delta^2) + \mathcal{O}(\sigma^2)\Big) \,\Big|\, r_q = r_k\Big] = \mathcal{O}\left(\frac{\mathcal{L}^2\Delta^2 + \sigma^2}{K^2}\right), \tag{36}$$

where the orders follow from $p_k = \Theta(1/K)$, $\mathrm{Attn}_k^{(t)} = \Theta(1)$, and $1 - \mathrm{Attn}_k^{(t)} = \Theta(1)$ during Phase I, and additionally $\mathrm{Attn}_{k'}^{(t)} = \Theta(1/K)$ for the cross-gradient.

Step 2: Evolution of bilinear weights. Using the update rule (27) and the gradient estimates (35)–(36), after T_1 steps:

$$A_k^{(T_1+1)} = A_k^{(0)} + \eta \sum_{t=0}^{T_1} \alpha_k^{(t)} = 0 + \eta \cdot \Theta\left(\frac{\mathcal{L}^2\Delta^2 + \sigma^2}{K}\right) \cdot T_1,$$

$$|B_{k,k'}^{(T_1+1)}| \le |B_{k,k'}^{(0)}| + \eta \sum_{t=0}^{T_1} |\beta_{k,k'}^{(t)}| = 0 + \eta \cdot \mathcal{O}\left(\frac{\mathcal{L}^2\Delta^2 + \sigma^2}{K^2}\right) \cdot T_1.$$

Setting $T_1 = \Theta\left(\frac{K\log(K)}{\eta(\mathcal{L}^2\Delta^2+\sigma^2)}\right)$, we obtain:

$$A_k^{(T_1+1)} = \Theta(\log K), \qquad |B_{k,k'}^{(T_1+1)}| = \mathcal{O}\left(\frac{\log K}{K}\right). \tag{37}$$

Step 3: Attention score at the end of Phase I. Using the definition of the attention score in (14) and the structure of the softmax:

$$\mathrm{Attn}_k^{(t)} = \frac{|\mathcal{R}_k|\, e^{r_k^\top Q^{(t)} r_k}}{\sum_{m\in[K]} |\mathcal{R}_m|\, e^{r_m^\top Q^{(t)} r_k}} = \frac{|\mathcal{R}_k|\, e^{A_k^{(t)}}}{\sum_{m\neq k} |\mathcal{R}_m|\, e^{B_{m,k}^{(t)}} + |\mathcal{R}_k|\, e^{A_k^{(t)}}} = \frac{1}{\sum_{m\neq k} \frac{|\mathcal{R}_m|}{|\mathcal{R}_k|} e^{B_{m,k}^{(t)} - A_k^{(t)}} + 1}.$$

From (37):

$$B_{m,k}^{(T_1+1)} - A_k^{(T_1+1)} = \mathcal{O}\left(\frac{\log K}{K}\right) - \Theta(\log K) = -\Theta(\log K),$$

so $\exp(B_{m,k}^{(T_1+1)} - A_k^{(T_1+1)}) \le \exp(-\Theta(\log K)) = \mathcal{O}(1/K)$.

Using $|\mathcal{R}_m|/|\mathcal{R}_k| = \Theta(1)$ from Lemma .1:

$$\mathrm{Attn}_k^{(T_1+1)} \ge \frac{1}{\mathcal{O}(1/K) \cdot \left(\frac{M}{|\mathcal{R}_k|} - 1\right) + 1} \ge \frac{1}{\mathcal{O}(1/K) \cdot \mathcal{O}(K) + 1} = \frac{1}{\mathcal{O}(1/u_k - 1/K) + 1},$$

where $u_k = (p_k - \rho/K) = \Theta(1/K)$ as defined in Lemma .1. Note that $1/u_k - 1/K = \Theta(\delta_0)$ where $\delta_0 = \mathcal{O}(1)$ is the constant defined in Assumption 2. Therefore:

$$\mathrm{Attn}_k^{(T_1+1)} = \Omega\left(\frac{1}{1 + \delta_0}\right),$$

completing the proof. □

C Proof of Proposition 4.4

Proof. The proof follows the same structure as Proposition 4.3, but now exploits the improved attention scores achieved at the end of Phase I.

Step 1: Gradient magnitudes in Phase II. At the beginning of Phase II ($t = T_1 + 1$), Proposition 4.3 ensures $\mathrm{Attn}_k^{(T_1+1)} = \Omega(1/(1+\delta_0))$, so $1 - \mathrm{Attn}_k^{(t)} = O(\delta_0)$ for $t \geq T_1 + 1$. Substituting into Lemma .2:

$$\alpha_k^{(t)} = \Theta\left(\frac{\delta_0^2\,\mathcal{L}^2\Delta^2 + \sigma^2}{K}\right), \tag{38}$$

$$|\beta_{k,k'}^{(t)}| = O\left(\frac{\delta_0^2\,\mathcal{L}^2\Delta^2 + \sigma^2}{K^2}\right), \tag{39}$$

where the factor δ_0^2 in the signal-dependent term reflects the reduced residual error $1 - \mathrm{Attn}_k^{(t)} = O(\delta_0)$.

Step 2: Evolution of bilinear weights. Starting from the Phase I endpoints in (37) and running for $T^* - T_1$ additional steps, by (27) and (38)–(39):

$$A_k^{(T^*)} = A_k^{(T_1+1)} + \eta \cdot \Theta\left(\frac{\delta_0^2\mathcal{L}^2\Delta^2 + \sigma^2}{K}\right) \cdot (T^* - T_1) = \Theta(\log(K\epsilon^{-1})),$$

$$|B_{k,k'}^{(T^*)}| \leq |B_{k,k'}^{(T_1+1)}| + \eta \cdot O\left(\frac{\delta_0^2\mathcal{L}^2\Delta^2 + \sigma^2}{K^2}\right) \cdot (T^* - T_1) = O\left(\frac{\log(K\epsilon^{-1})}{K}\right),$$

where $T^* = \Theta\left(\frac{K\log(K\epsilon^{-1})}{\eta(\delta_0^2\mathcal{L}^2\Delta^2+\sigma^2)}\right)$.

Step 3: Attention score at the end of Phase II. Following the same softmax calculation as in Step 3 of Appendix B:

$$B_{k,k'}^{(T^*)} - A_k^{(T^*)} = O\left(\frac{\log(K\epsilon^{-1})}{K}\right) - \Theta(\log(K\epsilon^{-1})) = -\Theta(\log(K\epsilon^{-1})),$$

so $\exp(B_{k,k'}^{(T^*)} - A_k^{(T^*)}) = O(\epsilon/K)$. Therefore:

$$\mathrm{Attn}_k^{(T^*)} \geq \frac{1}{O(\epsilon/K)\cdot(M/|\mathcal{R}_k| - 1) + 1} \geq \frac{1}{O(\epsilon)\cdot O(1/u_k - 1/K) + 1} = \Omega\left(\frac{1}{1+\delta_0\epsilon}\right),$$

which gives $1 - \mathrm{Attn}_k^{(T^*)} = O\left(\frac{\epsilon\delta_0}{1+\epsilon\delta_0}\right) = O(\epsilon)$, completing the proof. □

D Proof of Theorem 4.5

Proof. We combine the two-phase analysis from Propositions 4.3 and 4.4 with the loss characterization from Lemma 4.2.

Step 1: Attention score after T^* training steps. By Proposition 4.4, after $T^* = \Theta\left(\frac{K\log(K\epsilon^{-1})}{\eta(\delta_0^2\mathcal{L}^2\Delta^2+\sigma^2)}\right)$ gradient descent iterations, we have for every $k \in [K]$:

$$1 - \mathrm{Attn}_k^{(T^*)} = O(\epsilon\delta_0) = O(\epsilon), \tag{40}$$

where the last equality uses $\delta_0 = O(1)$.

Step 2: Bounding the prediction loss. Substituting (40) into the loss expression from Lemma 4.2, and conditioning on $r_q = r_k$:

$$l(\theta^{(T^*)}) = \sum_{k=1}^{K}\mathbb{E}\left[\mathbf{1}\{r_q = r_k\}\left(\left(1 - \mathrm{Attn}_k^{(T^*)}\right)^2\Theta(\mathcal{L}^2\Delta^2) + \sum_{m\in[K]}(\mathrm{Attn}_m^{(T^*)})^2\Theta(\sigma^2)\right)\right].$$

For the first term, using (40):

$$(1 - \mathrm{Attn}_k^{(T^*)})^2\,\Theta(\mathcal{L}^2\Delta^2) = O(\epsilon^2\delta_0^2)\cdot\Theta(\mathcal{L}^2\Delta^2) = O(\epsilon^2),$$

where the last step uses $\mathcal{L} = O(1/\Delta)$ (the condition in the theorem statement) and $\delta_0 = O(1)$, so that $\delta_0^2\mathcal{L}^2\Delta^2 = O(1)$.

For the second term, note that $\sum_{m\in[K]}(\mathrm{Attn}_m^{(T^*)})^2 \leq 1$ (since attention scores sum to 1 and each is at most 1), so:

$$\sum_{m\in[K]}(\mathrm{Attn}_m^{(T^*)})^2\,\Theta(\sigma^2) = \Theta(\sigma^2).$$

Combining and taking expectation over the query flow:

$$l(\theta^{(T^*)}) = \sum_{k=1}^{K} p_k \cdot \left(O(\epsilon^2) + \Theta(\sigma^2)\right) = O(\epsilon^2) + \Theta(\sigma^2),$$

where we used $\sum_k p_k = 1$.

Step 3: Sufficient data requirement. The condition $M \geq \mathrm{poly}(K)$ ensures that Lemma .1 holds with overwhelming probability, guaranteeing balanced representation of all K flow cases in the prompt. Specifically, we need $K^3 = O(M)$ so that the balanced data property $|\mathcal{R}_k| = \Theta(M/K)$ holds for all $k \in [K]$ simultaneously with probability at least $1 - 3\exp(-\Theta(M/K^2))$, which tends to 1 as M grows.

Combining the above three steps, we conclude that with at most $T^* = \Theta\left(\frac{K\log(K\epsilon^{-1})}{\eta(\delta_0^2\mathcal{L}^2\Delta^2+\sigma^2)}\right)$ iterations, the prediction loss satisfies $l(\theta^{(T^*)}) = O(\epsilon^2) + \Theta(\sigma^2)$. □

Proof of Corollary (Training Complexity). Each training iteration requires: (i) computing the embedding $E(P) \in \mathbb{R}^{(L+1)\times(M+1)}$; (ii) computing the softmax attention, which involves matrix multiplications of cost $O(M^2L)$; and (iii) a gradient step on $Q \in \mathbb{R}^{L\times L}$, also dominated by $O(M^2L)$. Multiplying by the number of iterations T^* gives the total complexity $O\left(M^2L \cdot \frac{K\log(K\epsilon^{-1})}{\eta(\delta_0^2\mathcal{L}^2\Delta^2+\sigma^2)}\right)$.

E Proof of Theorem 4.7

Proof. Let $\boldsymbol{x}^* = (x_1^*, \ldots, x_S^*)$ denote the global optimal rate vector satisfying the link capacity constraints in (1). By Theorem 4.5, the ICL prediction $\tilde{\boldsymbol{x}}_q$ satisfies $\mathbb{E}_{f\in\mathcal{F}}[(\tilde{x}_q^s - x_s^*)^2] = O(\epsilon^2) + \Theta(\sigma^2)$ for each flow $s \in [S]$. We now show that the projection $\hat{\boldsymbol{x}}_q$ in (9) preserves this bound.

Step 1: Non-expansiveness of Euclidean projection. Define the feasible set $\mathcal{X} := \{\boldsymbol{x} \in \mathbb{R}^S : \sum_{s\in S(\ell)} x_s \le c_\ell,\ \ell = 1, \ldots, L\}$. This is a closed convex polytope. The projection $\hat{\boldsymbol{x}}_q = \Pi_{\mathcal{X}}(\tilde{\boldsymbol{x}}_q)$ in (9) is the Euclidean projection onto $\mathcal{X}$. By the non-expansiveness property of Euclidean projection onto convex sets (e.g., [1, Proposition B.11]), for any $\boldsymbol{y} \in \mathcal{X}$:

$$\|\Pi_{\mathcal{X}}(\tilde{\boldsymbol{x}}_q) - \boldsymbol{y}\|_2 \le \|\tilde{\boldsymbol{x}}_q - \boldsymbol{y}\|_2. \tag{41}$$

Step 2: Applying to the optimum. Since $\boldsymbol{x}^* \in \mathcal{X}$ (the global optimum is feasible), setting $\boldsymbol{y} = \boldsymbol{x}^*$ in (41):

$$\|\hat{\boldsymbol{x}}_q - \boldsymbol{x}^*\|_2^2 = \|\Pi_{\mathcal{X}}(\tilde{\boldsymbol{x}}_q) - \boldsymbol{x}^*\|_2^2 \le \|\tilde{\boldsymbol{x}}_q - \boldsymbol{x}^*\|_2^2.$$

Step 3: Per-flow bound. Since $\|\hat{\boldsymbol{x}}_q - \boldsymbol{x}^*\|_2^2 = \sum_{s\in[S]}(\hat{x}_q^s - x_s^*)^2$, and $\|\tilde{\boldsymbol{x}}_q - \boldsymbol{x}^*\|_2^2 = \sum_{s\in[S]}(\tilde{x}_q^s - x_s^*)^2$, we have for each s:

$$(\hat{x}_q^s - x_s^*)^2 \le \|\hat{\boldsymbol{x}}_q - \boldsymbol{x}^*\|_2^2 \le \|\tilde{\boldsymbol{x}}_q - \boldsymbol{x}^*\|_2^2.$$

More precisely, by the coordinate-wise implication of the non-expansiveness result and the ICL prediction bound from Theorem 4.5:

$$\mathbb{E}_{f\in\mathcal{F}}[(\hat{x}_q^s - x_s^*)^2] \le \mathbb{E}_{f\in\mathcal{F}}[(\tilde{x}_q^s - x_s^*)^2] = O(\epsilon^2) + \Theta(\sigma^2),$$

completing the proof. □

F Proof of Proposition 4.8

Proof. The key observation is that after the prediction loss $l(\theta)$ has converged, the attention mechanism's behavior depends only on the *route vectors* $\{r_s\}$ (which determine the attention scores), not on the specific mapping function f. We formalize this below.

Step 1: Attention scores are function-independent. After convergence at $\theta^{(T^*)}$, the attention scores in (14) are:

$$\mathrm{Attn}_k^{(T^*)} = \frac{|\mathcal{R}_k|\exp(r_k^\top Q^{(T^*)} r_q)}{\sum_{m\in[K]}|\mathcal{R}_m|\exp(r_m^\top Q^{(T^*)} r_q)}.$$

These depend only on the trained parameter $Q^{(T^*)}$ and the route vectors $\{r_k\}$, not on the specific utility combination or mapping function. By Proposition 4.4, the converged attention score satisfies $\mathrm{Attn}_k^{(T^*)} = \Omega(1/(1+\delta_0\epsilon))$, i.e., $1 - \mathrm{Attn}_k^{(T^*)} = O(\epsilon)$, for all $k \in [K]$.

Step 2: Prediction error for an unseen function. For any unseen mapping function $f^u \in \mathcal{F}$ (possibly not encountered during training), if $r_q = r_k$, the ICL prediction using the prompt examples under f^u is:

$$\tilde{x}_q = \sum_{n\in[K]} \mathrm{Attn}_n^{(T^*)} f^u(r_n) + \sum_{s\in[M]} \mathrm{attn}_s^{(T^*)} \epsilon_s.$$

The squared prediction error is:

$$\mathbb{E}[(\tilde{x}_q - f^u(r_k))^2] = \left(\sum_{n\ne k} \mathrm{Attn}_n^{(T^*)}(f^u(r_n) - f^u(r_k))\right)^2 + \sum_{s\in[M]}(\mathrm{attn}_s^{(T^*)})^2\sigma^2.$$

By the triangle inequality and the Lipschitz condition $|f^u(r_n) - f^u(r_k)| \le \mathcal{L}\|r_n - r_k\|_2 = \Theta(\mathcal{L}\Delta)$:

$$\begin{aligned}\left|\sum_{n\ne k} \mathrm{Attn}_n^{(T^*)}(f^u(r_n) - f^u(r_k))\right| &\le \sum_{n\ne k} \mathrm{Attn}_n^{(T^*)}\,|f^u(r_n) - f^u(r_k)| \\ &\le (1 - \mathrm{Attn}_k^{(T^*)})\,\Theta(\mathcal{L}\Delta) = O(\epsilon)\cdot\Theta(\mathcal{L}\Delta).\end{aligned}$$

Squaring and using $\mathcal{L}\Delta\delta_0 = O(1)$ (from $\mathcal{L} = O(1/\Delta)$ and $\delta_0 = O(1)$):

$$\mathbb{E}[(\tilde{x}_q - f^u(r_k))^2] = O(\epsilon^2) + \Theta(\sigma^2).$$

Step 3: Projection preserves the bound. By the same non-expansiveness argument as in Theorem 4.7, the projected prediction $\hat{x}_q$ satisfies $\mathbb{E}[(\hat{x}_q - x_q^*)^2] \le \mathbb{E}[(\tilde{x}_q - x_q^*)^2] = O(\epsilon^2) + \Theta(\sigma^2)$.

This establishes the in-context learning capability: the converged transformer can generate accurate predictions for unseen mapping functions without fine-tuning, enabling zero-shot adaptation to dynamic network environments. □

Proof of Corollary (Inference Delay). At inference time, the transformer processes a new prompt P with M tokens through a single forward pass. The dominant cost is the self-attention computation: multiplying the $(L+1) \times M$ embedding matrix with the $M \times (M+1)$ attention weight matrix, which requires $O(M^2 L)$ operations. No gradient computation or weight update is needed.

G Proof of Theorem 4.10

Proof. We bound the utility loss ΔU defined in (17) by combining the Lipschitz continuity of the utility functions (Assumption 1) with the prediction accuracy established in Theorem 4.7 and Proposition 4.8.

Step 1: Lipschitz bound on utility loss. By Assumption 1, each utility function U_s is $\mathcal{K}$-Lipschitz continuous. Therefore:

$$U_s(x_s^*) - U_s(\hat{x}_s) \le |U_s(x_s^*) - U_s(\hat{x}_s)| \le \mathcal{K}\, |x_s^* - \hat{x}_s|.$$

Summing over all sources:

$$\Delta U = \frac{1}{S}\, \mathbb{E}_{f \in \mathcal{F}}\Big[\sum_{s \in [S]} \big(U_s(x_s^*) - U_s(\hat{x}_s)\big) \Big] \le \frac{\mathcal{K}}{S}\, \mathbb{E}_{f \in \mathcal{F}}\Big[\sum_{s \in [S]} |x_s^* - \hat{x}_s| \Big].$$

Step 2: From ℓ_1 to ℓ_2 norm. By the Cauchy–Schwarz inequality, $\sum_{s \in [S]} |x_s^* - \hat{x}_s| \le \sqrt{S}\, \|\boldsymbol{x}^* - \hat{\boldsymbol{x}}\|_2$. Substituting:

$$\Delta U \le \frac{\mathcal{K}}{\sqrt{S}}\, \mathbb{E}_{f \in \mathcal{F}}\big[\|\boldsymbol{x}^* - \hat{\boldsymbol{x}}\|_2 \big].$$

Step 3: Bounding the ℓ_2 error. By Jensen's inequality, $\mathbb{E}[\|\boldsymbol{x}^* - \hat{\boldsymbol{x}}\|_2] \le \sqrt{\mathbb{E}[\|\boldsymbol{x}^* - \hat{\boldsymbol{x}}\|_2^2]}$. From Theorem 4.7 and Proposition 4.8, each coordinate satisfies $\mathbb{E}[(\hat{x}_s - x_s^*)^2] = O(\epsilon^2) + \Theta(\sigma^2)$. Summing:

$$\mathbb{E}\big[\|\boldsymbol{x}^* - \hat{\boldsymbol{x}}\|_2^2\big] = \sum_{s \in [S]} \mathbb{E}[(\hat{x}_s - x_s^*)^2] = S \cdot \big(O(\epsilon^2) + \Theta(\sigma^2)\big).$$

Therefore:

$$\Delta U \le \frac{\mathcal{K}}{\sqrt{S}} \cdot \sqrt{S \cdot (O(\epsilon^2) + \Theta(\sigma^2))} = \mathcal{K}\sqrt{O(\epsilon^2) + \Theta(\sigma^2)} = O\big(\mathcal{K}\sqrt{\epsilon^2 + \sigma^2}\big),$$

which completes the proof. □

H Proof of Theorem 5.1

In this section, we prove the training convergence result for the insufficient sample case ($M < K$) stated in Section 5.2. For ease of reference, we first summarize the key notation, then proceed with the proof.

H.1 Notation Summary

- $Z := (g(r_1), \ldots, g(r_M)) \in \mathbb{R}^{N \times M}$: matrix of prompt representations.
- $\bar{Z} := (Z^\top Z + N\sigma^2 I_M)^{1/2} \in \mathbb{R}^{M \times M}$: regularized representation matrix.
- $z_q := g(r_q) \in \mathbb{R}^N$: representation of query flow.
- $x := (x_1, \ldots, x_M)^\top \in \mathbb{R}^M$: prompt labels (source rates).
- $s_q^h := \mathrm{softmax}(R^\top Q_h r_q) \in \mathbb{R}^M$: attention weights for head h.
- $D_q^{(t)} := (s_q^{1,(t)}, \ldots, s_q^{H,(t)}) \in \mathbb{R}^{M \times H}$: concatenated attention weights.
- $B_q^{(t)} := \bar{Z}\, D_q^{(t)} \in \mathbb{R}^{M \times H}$: projected attention matrix.
- $\bar{f}_{\max} := \sqrt{\bar{x}^2 + N\sigma^2}$: bound on column norms of $\bar{Z}$.
- $\zeta_0 := \lambda_{\min}(B_q^{(0)} B_q^{(0)\top}) > 0$: initial minimum eigenvalue.

H.2 Proof

Proof. The proof proceeds in several steps: (i) reformulating the loss, (ii) establishing a Polyak–Łojasiewicz (PL) condition, (iii) proving smoothness, and (iv) combining via an inductive argument.

Step 1: Loss function reformulation. We show that the loss function admits the decomposition:

$$l(\theta) = \frac{1}{2} \|\bar{Z}\, \delta_q^\theta\|_2^2 + l^*, \tag{42}$$

where $\delta_q^\theta := \sum_{h=1}^H V_h\, s_q^h - A_{:,q}$, with $A_{:,q} := (Z^\top Z + N\sigma^2 I_M)^{-1} Z^\top z_q$, and l^* is independent of θ.

To derive (42), let $\epsilon^P := (\epsilon_1, \ldots, \epsilon_M) \in \mathbb{R}^{N\times M}$, so that $x = (Z+\epsilon^P)^\top \lambda$. The transformer prediction is $\hat{x}_q = x^\top \hat{a}_q$ where $\hat{a}_q := \sum_{h=1}^H V_h\, s_q^h \in \mathbb{R}^M$, and the true label is $x_q^* = \lambda^\top (z_q + \epsilon_q)$. The loss is:

$$\begin{aligned} l(\theta) &= \frac{1}{2}\mathbb{E}_{\lambda,\epsilon}\left[(x^\top \hat{a}_q - \lambda^\top (z_q + \epsilon_q))^2\right] \\ &= \frac{1}{2}\mathbb{E}_{\lambda,\epsilon}\left[(\lambda^\top (Z + \epsilon^P)\hat{a}_q - \lambda^\top (z_q + \epsilon_q))^2\right]. \end{aligned}$$

Using Assumption 3 that $\mathbb{E}[\lambda\lambda^\top] = I_N$:

$$l(\theta) = \frac{1}{2}\mathbb{E}_{\epsilon}\left[\|(Z + \epsilon^P)\hat{a}_q - (z_q + \epsilon_q)\|_2^2\right].$$

Since $\mathbb{E}[\epsilon_s] = 0$ and $\mathbb{E}[\epsilon_s \epsilon_s^\top] = \sigma^2 I_N$, the cross-terms vanish, giving:

$$l(\theta) = \frac{1}{2}\left(\|Z\hat{a}_q - z_q\|_2^2 + N\sigma^2\|\hat{a}_q\|_2^2 + N\sigma^2\right).$$

Completing the square with $\bar{Z}^2 = Z^\top Z + N\sigma^2 I_M$:

$$\begin{aligned} \|Z\hat{a}_q - z_q\|_2^2 + N\sigma^2\|\hat{a}_q\|_2^2 &= \hat{a}_q^\top (Z^\top Z + N\sigma^2 I_M)\hat{a}_q - 2z_q^\top Z\hat{a}_q + \|z_q\|_2^2 \\ &= \|\bar{Z}(\hat{a}_q - A_{:,q})\|_2^2 + \|z_q\|_2^2 - z_q^\top Z(Z^\top Z + N\sigma^2 I_M)^{-1} Z^\top z_q. \end{aligned}$$

Setting $l^* := \frac{1}{2}(\|z_q\|_2^2 + N\sigma^2 - z_q^\top Z(Z^\top Z + N\sigma^2 I_M)^{-1} Z^\top z_q)$ yields (42).

Step 2: Bound on l^*. Using $(ZZ^\top + N\sigma^2 I_N)^{-1} \preceq \frac{1}{N\sigma^2} I_N$ and $\|z_q\|_2 \le \bar{x}$:

$$l^* = \frac{N\sigma^2}{2}\left(1 + z_q^\top (ZZ^\top + N\sigma^2 I_N)^{-1} z_q\right) \le \frac{N\sigma^2 + \bar{x}^2}{2}. \tag{43}$$

Step 3: Reparameterization. Define $\gamma := \sqrt{\eta_w/\eta_Q}$, $\alpha_{h,q} := V_h/\gamma$, and $\xi := \{Q_h, \alpha_{h,q}\}_{h=1}^H$, so the gradient descent updates become $\xi^{(t)} = \xi^{(t-1)} - \eta_Q \nabla_\xi l(\xi^{(t-1)})$. With this reparameterization, $\delta_q^\theta = \gamma \sum_h \alpha_{h,q}\, s_q^h - A_{:,q}$.

Step 4: PL condition. The gradient with respect to $\alpha_{h,q}$ is:

$$\frac{\partial l}{\partial \alpha_{h,q}} = \gamma\, (b_q^h)^\top \bar{Z}\, \delta_q^\theta,$$

where $b_q^h := \bar{Z}\, s_q^h$ is the h-th column of B_q. Collecting over all heads:

$$\|\nabla_{\alpha_q} l\|_2^2 = \gamma^2\, (\bar{Z}\, \delta_q^\theta)^\top B_q B_q^\top (\bar{Z}\, \delta_q^\theta) \ge \gamma^2\, \lambda_{\min}(B_q B_q^\top)\, \|\bar{Z}\, \delta_q^\theta\|_2^2.$$

We will show by induction (Step 6) that $\lambda_{\min}(B_q^{(t)} B_q^{(t)\top}) \ge \zeta_0/2$ for all t. Thus:

$$\|\nabla_\xi l(\xi^{(t)})\|_2^2 \ge \gamma^2 \cdot \frac{\zeta_0}{2} \cdot 2\left(l(\xi^{(t)}) - l^*\right) = \gamma^2 \zeta_0 \left(l(\xi^{(t)}) - l^*\right). \tag{44}$$

This is the PL condition with modulus $\mu_{\mathrm{PL}} = \gamma^2 \zeta_0$.

Step 5: Smoothness. The loss function $l(\xi)$ is $\mathcal{L}_s$-smooth in ξ. The smoothness constant depends on $\|\bar{Z}\|_2$, H, $\bar{f}_{\max}$, $\bar{r}$ (the bound on $\|r_k\|_2$), and ζ_0, and is given by:

$$\mathcal{L}_s = \Theta\left(\bar{r}^2 H^{3/4} \|\bar{Z}\|_2^5\, l(\theta^{(0)})\, (\bar{x}^2 + N\sigma^2)^{1/4}\, \zeta_0^{-9/4}\right).$$

This is obtained by bounding the Hessian of l with respect to ξ (see, e.g., [36] for an analogous derivation).

Step 6: Inductive argument. We prove by induction on t that for all $t \ge 0$:

(i) $|\alpha_{h,q}^{(t)}| \le \bar{\alpha} := \frac{\sqrt{2(l(\theta^{(0)}) - l^*)}}{4\|\bar{Z}\|_2 \gamma \zeta_0}$;
(ii) $\lambda_{\min}(B_q^{(t)} B_q^{(t)\top}) \ge \zeta_0/2$;
(iii) $l(\theta^{(t)}) - l^* \le (1 - \eta_w \zeta_0/2)^t\, (l(\theta^{(0)}) - l^*)$.

Base case ($t = 0$): Since $\alpha_{h,q}^{(0)} = V_h^{(0)}/\gamma = 0$ (zero initialization of value weights), all three conditions hold trivially.

Inductive step: Assume (i)–(iii) hold for all $s \in \{0, \ldots, t-1\}$.

Verifying (ii): The Lipschitz property of softmax and the gradient bounds give:

$$\|Q_h^{(t)} - Q_h^{(0)}\|_F \le \sum_{s=0}^{t-1} \eta_Q \left\|\frac{\partial l}{\partial Q_h}\right\|_F \le \frac{8\sqrt{2}\gamma\bar{\alpha}\bar{f}_{\max}}{\gamma^2\zeta_0}\sqrt{l(\theta^{(0)}) - l^*}.$$

Since $\|s_q^h(\theta) - s_q^h(\theta')\|_2 \le 2\bar{r}^2\|Q_h - Q_h'\|_F$ (softmax Lipschitz property) and $b_q^h = \bar{Z}\, s_q^h$:

$$\|B_q^{(t)} - B_q^{(0)}\|_F \le 2\|\bar{Z}\|_2\sqrt{H}\max_h \|Q_h^{(t)} - Q_h^{(0)}\|_F.$$

By the choice of γ in the theorem statement, this is bounded by $(1-1/\sqrt{2})\sqrt{\zeta_0}$, which by Weyl's inequality ensures:

$$\lambda_{\min}(B_q^{(t)} B_q^{(t)\top}) \geq \zeta_0 - 2\|B_q^{(t)} - B_q^{(0)}\|_F \sqrt{\zeta_0} \geq \zeta_0/2.$$

Verifying (iii): By the smoothness of l and the condition $\eta_Q \leq 1/\mathcal{L}_s$:

$$l(\xi^{(t)}) \leq l(\xi^{(t-1)}) - \frac{\eta_Q}{2}\|\nabla_\xi l(\xi^{(t-1)})\|_2^2.$$

Substituting the PL condition (44):

$$l(\xi^{(t)}) - l^* \leq \left(1 - \frac{\eta_Q \gamma^2 \zeta_0}{2}\right)(l(\xi^{(t-1)}) - l^*) = \left(1 - \frac{\eta_w \zeta_0}{2}\right)(l(\xi^{(t-1)}) - l^*).$$

Verifying (i): From $|\partial l/\partial \alpha_{h,q}| \leq \gamma \|\bar{Z}\|_2 \sqrt{2(l-l^*)}$:

$$|\alpha_{h,q}^{(t)}| \leq \sum_{s=0}^{t-1} \eta_Q \left|\frac{\partial l}{\partial \alpha_{h,q}}\right| \leq \frac{4\|\bar{Z}\|_2}{\gamma \zeta_0}\sqrt{2(l(\theta^{(0)}) - l^*)} = \bar{\alpha}.$$

Step 7: Conclusion. From (iii), as $t \to \infty$, $l(\theta^{(t)}) \to l^*$. By the bound (43):

$$\lim_{t\to\infty} l(\theta^{(t)}) \leq l^* \leq \frac{N\sigma^2 + \bar{x}^2}{2}.$$

By the Lipschitz continuity of the utility functions (Assumption 1), the utility loss satisfies:

$$\lim_{t\to\infty} \Delta U^{(t)} \leq \mathcal{O}\left(\mathcal{K}\sqrt{N\sigma^2 + \bar{x}^2}\right),$$

using the same Lipschitz-to-ℓ_2-to-loss chain as in the proof of Theorem 4.10 (Appendix G). □

I Proof of Theorem 5.2

Proof. We show that the trained transformer's output converges to the ridge regression solution, which in turn provides near-optimal rate allocations. The proof has three main parts: (i) establishing the connection between the transformer's target and ridge regression, (ii) bounding the prediction error, and (iii) deriving the required number of training steps.

Step 1: Ridge regression solution and its connection to $A_{:,q}$. The ridge regression problem has the closed-form solution:

$$\hat{\lambda} = (ZZ^\top + N\sigma^2 I_N)^{-1} Z x.$$

The ridge prediction for the query flow is $x_q^\star = \hat{\lambda}^\top z_q$. We now show that $x_q^\star = x^\top A_{:,q}$.

By the Sherman–Morrison–Woodbury identity, $Z(Z^\top Z + N\sigma^2 I_M)^{-1} Z^\top = ZZ^\top (ZZ^\top + N\sigma^2 I_N)^{-1}$. Therefore:

$$\begin{aligned} x^\top A_{:,q} &= x^\top (Z^\top Z + N\sigma^2 I_M)^{-1} Z^\top z_q = (Z x)^\top (ZZ^\top + N\sigma^2 I_N)^{-1} z_q \\ &= z_q^\top (ZZ^\top + N\sigma^2 I_N)^{-1} Z x = \hat{\lambda}^\top z_q = x_q^\star. \end{aligned}$$

Step 2: Prediction error decomposition. The transformer's prediction is $\hat{x}_q = x^\top \hat{a}_q$ where $\hat{a}_q = \sum_h V_h s_q^h$. The error relative to the ridge prediction is:

$$|\hat{x}_q - x_q^\star| = |x^\top(\hat{a}_q - A_{:,q})| = |x^\top \delta_q^{\theta^{(T)}}| \leq \|x\|_2 \, \|\delta_q^{\theta^{(T)}}\|_2,$$

by the Cauchy–Schwarz inequality.

Step 3: Bounding $\|\delta_q^{\theta^{(T)}}\|_2$. From the loss reformulation (42):

$$l(\theta^{(T)}) - l^* = \frac{1}{2}\|\bar{Z}\,\delta_q^{\theta^{(T)}}\|_2^2 \geq \frac{N\sigma^2}{2}\|\delta_q^{\theta^{(T)}}\|_2^2,$$

since $\lambda_{\min}(\bar{Z}^\top \bar{Z}) = \lambda_{\min}(Z^\top Z + N\sigma^2 I_M) \geq N\sigma^2$. By the training convergence result from Theorem 5.1:

$$l(\theta^{(T)}) - l^* \leq \left(1 - \frac{\eta_w \zeta_0}{2}\right)^T (l(\theta^{(0)}) - l^*).$$

Combining:

$$\|\delta_q^{\theta^{(T)}}\|_2 \leq \sqrt{\frac{2(l(\theta^{(0)}) - l^*)}{N\sigma^2}} \left(1 - \frac{\eta_w \zeta_0}{2}\right)^{T/2}. \tag{45}$$

Step 4: Bounding $\|x\|_2$ with high probability. Recall $x_s = \lambda^\top(g(r_s) + \epsilon_s)$, so $x = (Z + \epsilon^P)^\top \lambda$. Thus $\|x\|_2 \leq \|Z\|_2\|\lambda\|_2 + \|(\epsilon^P)^\top \lambda\|_2$. For the noise term, $(\epsilon^P)^\top \lambda = \sum_{s=1}^M (\epsilon_s^\top \lambda)\, e_s$ where each $\epsilon_s^\top \lambda$ has variance $\sigma^2\|\lambda\|_2^2$. By the Laurent–Massart tail bound, with probability at least $1-\delta$:

$$\|(\epsilon^P)^\top \lambda\|_2^2 \leq \|\lambda\|_2^2 \sigma^2 \big(M + 2\sqrt{M\log(1/\delta)} + 2\log(1/\delta)\big).$$

Denoting $C := \|Z\|_2 + \sqrt{\sigma^2(M + 2\sqrt{M\log(1/\delta)} + 2\log(1/\delta))}$, we have $\|x\|_2 \le \|\lambda\|_2 \cdot C$ with probability at least $1-\delta$. Under the boundedness condition $\|\lambda\|_2 \le B$ at inference time, $\|x\|_2 \le BC$.

Step 5: Combining the bounds. From Steps 2–4:

$$\begin{aligned}\frac{1}{2}(\hat{x}_q - x_q^\star)^2 &\le \frac{1}{2}\|x\|_2^2\,\|\delta_q^{\theta^{(T)}}\|_2^2 \\ &\le \frac{B^2C^2(l(\theta^{(0)}) - l^*)}{N\sigma^2}\left(1 - \frac{\eta_w\zeta_0}{2}\right)^T.\end{aligned}$$

Setting this $\le \varepsilon$ and solving for T:

$$T \ge \frac{\log\left(\frac{B^2C^2(l(\theta^{(0)})-l^*)}{N\sigma^2\varepsilon}\right)}{\log\left(\frac{1}{1-\eta_w\zeta_0/2}\right)}.$$

Since $\log(1/(1-x)) \approx x$ for small x and $C^2 = O(\|Z\|_2^2 + \sigma^2\log(1/\delta))$:

$$T = O\left(\frac{1}{\eta_w\zeta_0}\log\left(\frac{B^2(\|Z\|_2^2 + \sigma^2\log(1/\delta))(l(\theta^{(0)}) - l^*)}{N\sigma^2\varepsilon}\right)\right) = \Omega\left(\log\left(\frac{\log(1/\delta)}{\varepsilon}\right)\right).$$

This establishes that with probability at least $1-\delta$:

$$\frac{1}{2}(\hat{x}_q - x_q^\star)^2 \le \varepsilon, \qquad \text{where } x_q^\star = \hat{\lambda}^\top z_q.$$

Applying the Lipschitz bound (Assumption 1):

$$U(x_q^{\text{opt}}) - U(\hat{x}_q) \le \mathcal{K}\,|x_q^{\text{opt}} - \hat{x}_q| \le \mathcal{K}\big(|x_q^{\text{opt}} - x_q^\star| + |x_q^\star - \hat{x}_q|\big) \le O(\mathcal{K}\varepsilon),$$

where the last inequality uses $|x_q^\star - \hat{x}_q| \le \sqrt{2\varepsilon}$ from the above bound and $|x_q^{\text{opt}} - x_q^\star|$ is the irreducible ridge regression estimation error. This completes the proof. □

Remark on irreducible error. The gap between the ridge regression solution $x_q^\star = \hat{\lambda}^\top z_q$ and the true optimal label $x_q = \lambda^\top z_q$ arises from: (a) *bias* due to ridge regularization shrinking $\hat{\lambda}$ toward zero; and (b) *variance* from noise propagation in the prompt labels. This irreducible error is captured by the optimal loss $l^* = \frac{N\sigma^2}{2}(1 + z_q^\top(ZZ^\top + N\sigma^2 I_N)^{-1}z_q)$, which is nonzero whenever $\sigma^2 > 0$ or the prompt examples do not perfectly span the representation space.